

REDES NEURONALES

En esta actividad vamos a utilizar una red neuronal para clasificar imágenes de dígitos del 0 al 9 escritos a mano. Para ello, utilizaremos Keras con TensorFlow.

El dataset a utilizar es MNIST, una base de datos constituida por (como no) imágenes de dígitos escritos a mano. Este dataset es ampliamente utilizado en docencia como punto de entrada al entrenamiento de redes neuronales y otros, pero también es muy utilizado en trabajos reales de investigación para el entrenamiento de imágenes. Puedes consultar más información sobre el dataset en [este enlace](#).

El código utilizado para contestar tiene que quedar claramente reflejado en el Notebook. Puedes crear nuevas celdas si así lo deseas para estructurar tu código y sus salidas. A la hora de entregar el notebook, **asegúrate de que los resultados de ejecutar tu código han quedado guardados y que son perfectamente visibles en la versión PDF que debes entregar adjunta**. Por ejemplo, a la hora de entrenar una red neuronal tiene que verse claramente un log de los resultados de cada epoch.

```
In [1]: from keras.datasets.mnist import load_data
import tensorflow as tf
import matplotlib.pyplot as plt
```

Tenemos la suerte de que el dataset MNIST, el que vamos a utilizar en esta actividad, está guardado en Keras, por lo que podemos utilizarlo sin necesidad de buscar el dataset de forma externa.

```
In [2]: mnist = tf.keras.datasets.fashion_mnist
```

Llamar a **load_data** en este dataset nos dará dos conjuntos de dos listas, estos serán los valores de entrenamiento y prueba para los gráficos que contienen los dígitos y sus etiquetas.

Nota: Aunque en esta actividad lo veis de esta forma, también lo vais a poder encontrar como 4 variables de esta forma: training_images, training_labels, test_images, test_labels = mnist.load_data()

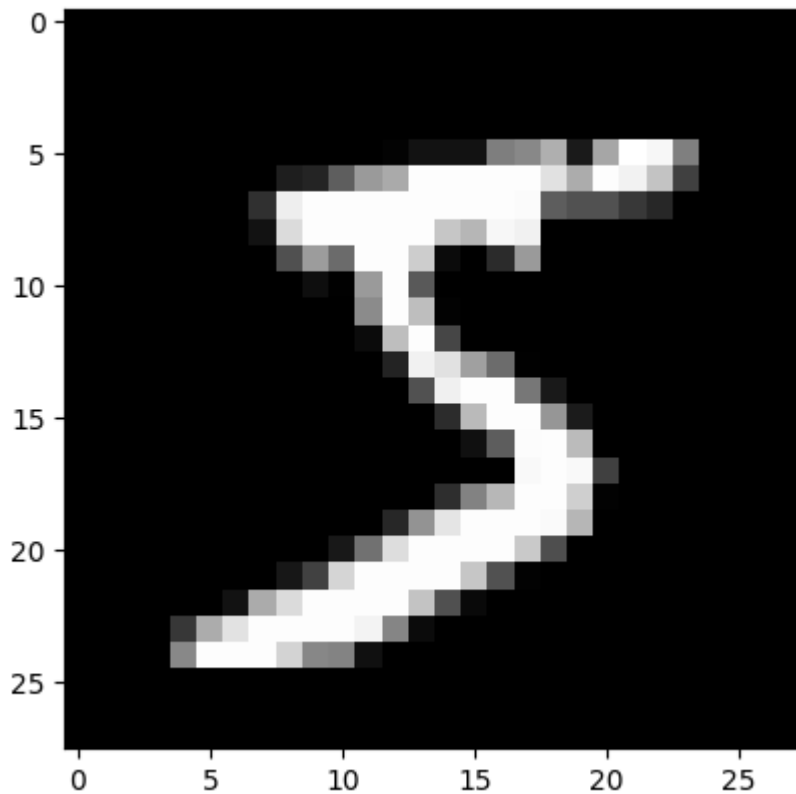
```
In [3]: (training_images, training_labels), (test_images, test_labels) =
load_data()
```

Antes de continuar vamos a dar un vistazo a nuestro dataset, para ello vamos a ver una imagen de entrenamiento y su etiqueta o clase.

```
In [4]: import numpy as np
np.set_printoptions(linewidth=200)
plt.imshow(training_images[0], cmap="gray") # recordad que siempre es
preferible trabajar en blanco y negro
#
```

```
print(training_labels[0])
print(training_images[0])
```

[illegible]



1. Información sobre el dataset

Una vez tenemos los datos cargados en memoria, vamos a obtener información sobre los mismos.

****Pregunta 1.1 (0.25 puntos)**** ¿Cuántas imágenes hay de *training* y de *test*? ¿Qué tamaño tienen las imágenes?

```
In [5]: # Si falta alguna dependencia en el entorno local, se puede ejecutar una
        # vez:
        # %pip install tensorflow numpy pandas matplotlib scikit-learn

import random
import time
import pandas as pd
from tensorflow import keras
from tensorflow.keras import layers, regularizers
from sklearn.metrics import confusion_matrix, classification_report

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

# Guardo copias antes de transformar los datos para poder consultar el
# formato original.
training_images_raw = training_images.copy()
test_images_raw = test_images.copy()
training_labels_raw = training_labels.copy()
test_labels_raw = test_labels.copy()

print("Imágenes de training:", training_images.shape[0])
```

```
print("Imágenes de test:", test_images.shape[0])
print("Tamaño de cada imagen de training:", training_images.shape[1:])
print("Tamaño de cada imagen de test:", test_images.shape[1:])
print("Tipo de dato de X:", training_images.dtype)
print("Tipo de dato de y:", training_labels.dtype)
```

Imágenes de training: 60000

Imágenes de test: 10000

Tamaño de cada imagen de training: (28, 28)

Tamaño de cada imagen de test: (28, 28)

Tipo de dato de X: uint8

Tipo de dato de y: uint8

Hay **60.000 imágenes de entrenamiento** y **10.000 imágenes de test**. Cada imagen tiene un tamaño de **28 x 28 píxeles**, por lo que cada ejemplo contiene 784 valores de intensidad antes de aplicar ningún preprocesado. En este dataset original, esos valores son números enteros entre 0 y 255.

****Pregunta 1.2 *(0.25 puntos)**** Realizar una exploración de las variables que contienen los datos. Describir en qué consiste un example del dataset (qué información se guarda en cada imagen) y describir qué contiene la información en y.

```
In [6]: print("Rango de valores en X_train:", training_images_raw.min(), "-",
          training_images_raw.max())
          print("Rango de valores en X_test:", test_images_raw.min(), "-",
          test_images_raw.max())
          print("Forma de una imagen individual:", training_images_raw[0].shape)
          print("Etiqueta asociada a esa imagen:", training_labels_raw[0])

          label_counts = pd.Series(training_labels_raw).value_counts().sort_index()
          display(pd.DataFrame({"dígito": label_counts.index, "n_imágenes_training":
          label_counts.values}))

          fig, axes = plt.subplots(2, 5, figsize=(10, 4))
          for digit, ax in enumerate(axes.ravel()):
              idx = np.where(training_labels_raw == digit)[0][0]
              ax.imshow(training_images_raw[idx], cmap="gray")
              ax.set_title(f"Dígito {digit}")
              ax.axis("off")
          plt.tight_layout()
          plt.show()
```

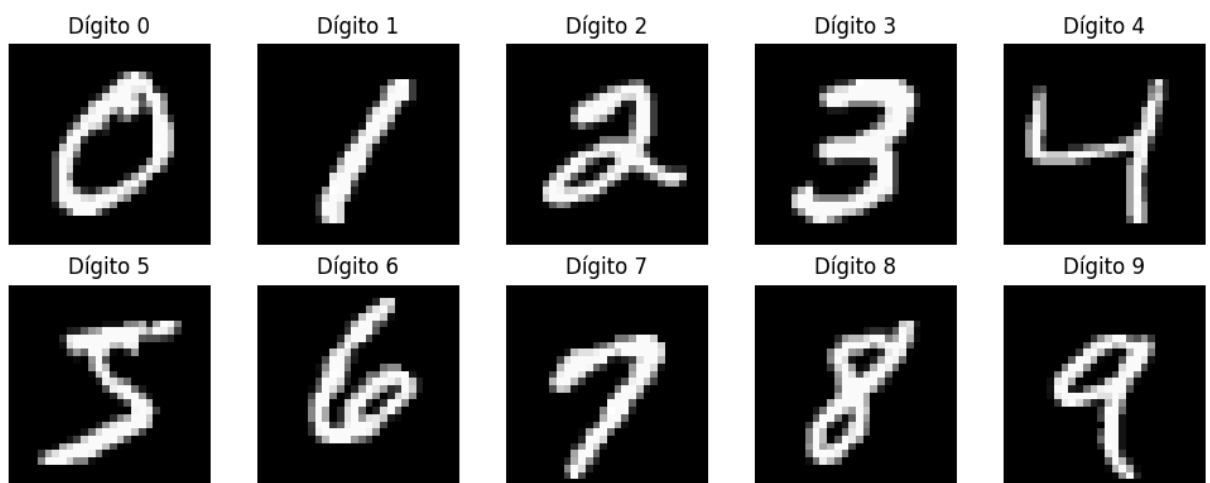
Rango de valores en X_train: 0 - 255

Rango de valores en X_test: 0 - 255

Forma de una imagen individual: (28, 28)

Etiqueta asociada a esa imagen: 5

	dígito	n_imágenes_training
0	0	5923
1	1	6742
2	2	5958
3	3	6131
4	4	5842
5	5	5421
6	6	5918
7	7	6265
8	8	5851
9	9	5949



Cada ejemplo del dataset es una imagen en escala de grises de 28 x 28 píxeles. La variable `X` guarda una matriz por imagen; cada posición de esa matriz representa la intensidad de un píxel. La variable `y` contiene la clase correcta de cada imagen, es decir, un número entero entre 0 y 9 que indica qué dígito manuscrito aparece.

2. Normalización y preprocesado de los datos

Pregunta 2.1 (0.25 puntos) Habreis notado que todos los valores numericos están entre 0 y 255. Si estamos entrenando una red neuronal, una buena practica es transformar todos los valores entre 0 y 1, un proceso llamado "normalización" y afortunadamente en Python es fácil normalizar una lista. ¿Cómo lo podemos hacer?

```
In [7]: training_images = training_images.astype("float32") / 255.0
test_images = test_images.astype("float32") / 255.0

print("Rango tras normalizar X_train:", training_images.min(), "-",
      training_images.max())
print("Rango tras normalizar X_test:", test_images.min(), "-",
      test_images.max())
```

Rango tras normalizar X_train: 0.0 – 1.0

Rango tras normalizar X_test: 0.0 – 1.0

Pregunta 2.2 (0.25 puntos) Utiliza la función **reshape** de Numpy para convertir las imágenes en vectores de características de un tamaño de (N, 784). Explica con tus palabras por qué es necesario hacer esto.

```
In [8]: X_train = training_images.reshape((training_images.shape[0], 28 * 28))
X_test = test_images.reshape((test_images.shape[0], 28 * 28))

print("Forma original de entrenamiento:", training_images.shape)
print("Forma vectorizada de entrenamiento:", X_train.shape)
print("Forma original de test:", test_images.shape)
print("Forma vectorizada de test:", X_test.shape)
```

Forma original de entrenamiento: (60000, 28, 28)

Forma vectorizada de entrenamiento: (60000, 784)

Forma original de test: (10000, 28, 28)

Forma vectorizada de test: (10000, 784)

Respuesta a la pregunta 2.2:

Para un perceptrón multicapa con capas **Dense**, cada ejemplo debe entrar como un vector de características. Una imagen de 28 x 28 píxeles se transforma en un vector de 784 posiciones para que cada neurona de la primera capa densa pueda recibir todos los píxeles como variables de entrada. Si se utilizara una red convolucional, no haría falta aplanar manualmente las imágenes, pero en esta actividad el objetivo es trabajar con un MLP.

Pregunta 2.3 (0.25 puntos) Para facilitar el desarrollo de la actividad, vamos a expresar las etiquetas así:

```
In [9]: training_labels = tf.keras.utils.to_categorical(training_labels)
test_labels = tf.keras.utils.to_categorical(test_labels)
```

Muestra cómo son ahora los datos, como resultado de este cambio y también de los realizados en las dos preguntas anteriores. Debate cómo se beneficiará la red neuronal de todos estos cambios.

```
In [10]: y_train = training_labels
y_test = test_labels

print("X_train:", X_train.shape, X_train.dtype)
print("X_test:", X_test.shape, X_test.dtype)
print("y_train:", y_train.shape, y_train.dtype)
print("y_test:", y_test.shape, y_test.dtype)
print("Etiqueta original del primer ejemplo:", training_labels_raw[0])
print("Etiqueta one-hot del primer ejemplo:", y_train[0])
print("Primeros 12 valores normalizados del primer vector:")
print(X_train[0][:12])
```

```
X_train: (60000, 784) float32
X_test: (10000, 784) float32
y_train: (60000, 10) float64
y_test: (10000, 10) float64
Etiqueta original del primer ejemplo: 5
Etiqueta one-hot del primer ejemplo: [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
Primeros 12 valores normalizados del primer vector:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
```

Respuesta a la pregunta 2.3:

Después del preprocesado, cada imagen queda como un vector de 784 características con valores entre 0 y 1, y cada etiqueta queda como un vector one-hot de 10 posiciones. La normalización ayuda a que el entrenamiento sea más estable, porque todos los píxeles quedan en una escala parecida. La vectorización adapta las imágenes a las capas densas y el one-hot encoding permite usar `categorical_crossentropy`, comparando las probabilidades predichas por `softmax` con la clase real.

3. Creación del Modelo

Ahora vamos a definir el modelo, pero antes vamos a repasar algunos comandos y conceptos muy útiles:

- **Sequential:** Eso define una SECUENCIA de capas en la red neuronal
- **Dense:** Añade una capa de neuronas
- **Flatten:** ¿Recuerdas cómo eran las imágenes cuando las imprimiste para poder verlas? Un cuadrado, Flatten toma ese cuadrado y lo convierte en un vector de una dimensión.

Cada capa de neuronas necesita una función de activación. Normalmente se usa la función relu en las capas intermedias y softmax en la ultima capa (en problemas de clasificación de más de dos items)

- **Relu** significa que "Si $X > 0$ devuelve X , si no, devuelve 0", así que lo que hace es pasar sólo valores 0 o mayores a la siguiente capa de la red.
- **Softmax** toma un conjunto de valores, y escoge el más grande.

Pregunta 3.1 (0.5 puntos). Utilizando Keras, y preparando los datos de X e Y como fuera necesario, define y entrena una red neuronal que sea capaz de clasificar imágenes de MNIST con las siguientes características:

- Una capa de entrada del tamaño adecuado.
- Una capa oculta de 512 neuronas.
- Una capa final con 10 salidas.

```
In [11]: def reset_seeds(seed=SEED):
          """Reinicia las semillas para que las comparaciones sean más reproducibles."""
          random.seed(seed)
          np.random.seed(seed)
          tf.random.set_seed(seed)
```

```

# Esta función evita repetir mucho código en los apartados comparativos.
# Mantiene una misma base de MLP y permite cambiar solo el elemento que se
# quiere estudiar.
def create_mlp(
    hidden_units=512,
    activation="relu",
    output_units=10,
    initializer="glorot_uniform",
    input_dim=784,
    hidden_layers=None,
    dropout_rate=0.0,
    l2_value=0.0,
    batch_norm=False,
):
    if hidden_layers is None:
        hidden_layers = [hidden_units]

    model_layers = [layers.Input(shape=(input_dim,))]
    for units in hidden_layers:
        model_layers.append(
            layers.Dense(
                units,
                kernel_initializer=initializer,
                kernel_regularizer=regularizers.l2(l2_value) if l2_value
            else None,
                use_bias=not batch_norm,
            )
        )
    if batch_norm:
        model_layers.append(layers.BatchNormalization())
        model_layers.append(layers.Activation(activation))
    if dropout_rate:
        model_layers.append(layers.Dropout(dropout_rate))
    model_layers.append(layers.Dense(output_units, activation="softmax"))
    return keras.Sequential(model_layers)

model = create_mlp(hidden_units=512)
print("Modelo base creado con entrada de 784 características, capa oculta
de 512 neuronas y salida de 10 clases.")

```

Modelo base creado con entrada de 784 características, capa oculta de 512 neuronas y salida de 10 clases.

Pregunta 3.2 (0.25 puntos): ¿crees conveniente utilizar una capa flatten en este caso? Motiva tu respuesta.

```

In [12]: model_with_flatten = keras.Sequential([
    layers.Input(shape=(28, 28)),
    layers.Flatten(),
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax"),
])

print("Modelo alternativo con Flatten si se alimentan imágenes 28x28
directamente:")
model_with_flatten.summary()

```


Modelo alternativo con Flatten si se alimentan imágenes 28x28 directamente:
Model: "sequential_1"

Layer (type) Param #	Output Shape	
flatten (Flatten) 0	(None, 784)	
dense_2 (Dense) 401,920	(None, 512)	
dense_3 (Dense) 5,130	(None, 10)	

Total params: 407,050 (1.55 MB)
Trainable params: 407,050 (1.55 MB)
Non-trainable params: 0 (0.00 B)

Respuesta a la pregunta 3.2:

En el modelo principal no es necesario incluir una capa `Flatten`, porque en el apartado anterior ya se ha aplicado `reshape` y cada imagen entra como un vector de 784 valores. Sí sería conveniente usar `Flatten` si quisiéramos pasar al modelo las imágenes con su forma original `(28, 28)`, ya que una capa `Dense` necesita recibir un vector por cada ejemplo.

Pregunta 3.3 (0.25 puntos): Utiliza la función `summary()` para mostrar la estructura de tu modelo.

```
In [13]: model.summary()
```

Model: "sequential"

Layer (type) Param #	Output Shape	
dense (Dense) 401,920	(None, 512)	
activation (Activation) 0	(None, 512)	
dense_1 (Dense) 5,130	(None, 10)	

Total params: 407,050 (1.55 MB)

Trainable params: 407,050 (1.55 MB)

Non-trainable params: 0 (0.00 B)

4: Compilación y entrenamiento

Pregunta 4.1 (0.5 puntos): Compila tu modelo. Utiliza ***categorical_crossentropy*** como función de pérdida, ***Adam*** como optimizador, y monitoriza la ***tasa de acierto*** durante el entrenamiento. Explica qué hace cada cosa en la compilación.

```
In [14]: model.compile(
            optimizer=keras.optimizers.Adam(),
            loss="categorical_crossentropy",
            metrics=["accuracy"],
        )
        print("Modelo compilado.")
```

Modelo compilado.

Respuesta a la pregunta 4.1:

La función de pérdida `categorical_crossentropy` mide la diferencia entre la etiqueta real en formato one-hot y las probabilidades que produce `softmax`; por eso encaja con un problema multiclase como MNIST. El optimizador `Adam` ajusta el aprendizaje de forma adaptativa y suele funcionar bien sin tener que probar demasiados valores iniciales. La métrica `accuracy` indica qué proporción de imágenes se clasifican correctamente, así que permite seguir el entrenamiento de una forma bastante intuitiva.

Pregunta 4.2 (0.5 puntos): Utiliza la función ***fit()*** para entrenar tu modelo. Para ayudarte en tu primer entrenamiento, utiliza estos valores:

- epochs = 5
- batch_size = 32
- validation_split = 0.25

```
In [15]: def plot_history(history, title):
            hist = pd.DataFrame(history.history)
            fig, axes = plt.subplots(1, 2, figsize=(12, 4))
            axes[0].plot(hist["loss"], label="train")
            axes[0].plot(hist["val_loss"], label="validation")
            axes[0].set_title(f"{title} - loss")
            axes[0].set_xlabel("epoch")
            axes[0].legend()
            axes[1].plot(hist["accuracy"], label="train")
            axes[1].plot(hist["val_accuracy"], label="validation")
            axes[1].set_title(f"{title} - accuracy")
            axes[1].set_xlabel("epoch")
            axes[1].legend()
            plt.tight_layout()
            plt.show()

        def train_model(model_to_train, epochs=5, batch_size=32,
            validation_split=0.25, optimizer=None, verbose=0, callbacks=None):
            if optimizer is None:
```

```

        optimizer = keras.optimizers.Adam()
        model_to_train.compile(optimizer=optimizer,
                                loss="categorical_crossentropy", metrics=["accuracy"])
        start = time.time()
        history = model_to_train.fit(
            X_train,
            y_train,
            epochs=epochs,
            batch_size=batch_size,
            validation_split=validation_split,
            verbose=verbose,
            callbacks=callbacks,
        )
        elapsed = time.time() - start
        test_loss, test_acc = model_to_train.evaluate(X_test, y_test,
                                                       verbose=0)
        return history, elapsed, test_loss, test_acc

history_base, time_base, test_loss_base, test_acc_base = train_model(
    model, epochs=5, batch_size=32, validation_split=0.25, verbose=1
)
plot_history(history_base, "Modelo base 512 neuronas")

pd.DataFrame([
    {"modelo": "512 neuronas, 5 epochs",
     "val_accuracy_final": history_base.history["val_accuracy"][-1],
     "val_loss_final": history_base.history["val_loss"][-1],
     "test_accuracy": test_acc_base,
     "test_loss": test_loss_base,
     "tiempo_s": time_base,
    }])

```

Epoch 1/5

1/1407		10:38	454ms/step - accuracy: 0.0000e+00 - loss: 2.4166
30/1407		2s	2ms/step - accuracy: 0.5014 - loss: 1.6330
59/1407		2s	2ms/step - accuracy: 0.6126 - loss: 1.3054
88/1407		2s	2ms/step - accuracy: 0.6697 - loss: 1.1230
114/1407		2s	2ms/step - accuracy: 0.7028 - loss: 1.0155
142/1407		2s	2ms/step - accuracy: 0.7279 - loss: 0.9322
169/1407		2s	2ms/step - accuracy: 0.7464 - loss: 0.8699
194/1407		2s	2ms/step - accuracy: 0.7601 - loss: 0.8234
221/1407		2s	2ms/step - accuracy: 0.7722 - loss: 0.7821
248/1407		2s	2ms/step - accuracy: 0.7824 - loss: 0.7471
273/1407		2s	2ms/step - accuracy: 0.7907 - loss: 0.7193
299/1407		2s	2ms/step - accuracy: 0.7981 - loss: 0.6940
324/1407		2s	2ms/step - accuracy: 0.8045 - loss: 0.6726

350/1407		2s 2ms/step	accuracy: 0.8104	loss: 0.6525
376/1407		1s 2ms/step	accuracy: 0.8157	loss: 0.6344
401/1407		1s 2ms/step	accuracy: 0.8203	loss: 0.6184
427/1407		1s 2ms/step	accuracy: 0.8247	loss: 0.6033
451/1407		1s 2ms/step	accuracy: 0.8284	loss: 0.5905
476/1407		1s 2ms/step	accuracy: 0.8320	loss: 0.5783
499/1407		1s 2ms/step	accuracy: 0.8351	loss: 0.5679
525/1407		1s 2ms/step	accuracy: 0.8383	loss: 0.5568
535/1407		1s 2ms/step	accuracy: 0.8394	loss: 0.5528
540/1407		1s 2ms/step	accuracy: 0.8400	loss: 0.5508
544/1407		1s 2ms/step	accuracy: 0.8404	loss: 0.5493
548/1407		1s 2ms/step	accuracy: 0.8409	loss: 0.5477
551/1407		2s 2ms/step	accuracy: 0.8412	loss: 0.5466
553/1407		2s 3ms/step	accuracy: 0.8414	loss: 0.5458
555/1407		2s 3ms/step	accuracy: 0.8417	loss: 0.5451
565/1407		2s 3ms/step	accuracy: 0.8427	loss: 0.5413
584/1407		2s 3ms/step	accuracy: 0.8447	loss: 0.5345
604/1407		2s 3ms/step	accuracy: 0.8467	loss: 0.5276
626/1407		2s 3ms/step	accuracy: 0.8488	loss: 0.5204
650/1407		2s 3ms/step	accuracy: 0.8509	loss: 0.5129
674/1407		1s 3ms/step	accuracy: 0.8530	loss: 0.5058
698/1407		1s 3ms/step	accuracy: 0.8550	loss: 0.4990
724/1407		1s 3ms/step	accuracy: 0.8570	loss: 0.4920
746/1407		1s 3ms/step	accuracy: 0.8586	loss: 0.4864
773/1407		1s 3ms/step	accuracy: 0.8605	loss: 0.4797
791/1407		1s 3ms/step	accuracy: 0.8618	loss: 0.4755
808/1407		1s 3ms/step	accuracy: 0.8629	loss: 0.4716
816/1407		1s 3ms/step	accuracy: 0.8634	loss: 0.4698
822/1407		1s 3ms/step	accuracy: 0.8638	loss: 0.4685
840/1407		1s 3ms/step	accuracy: 0.8649	loss: 0.4646

857/1407		1s 3ms/step	- accuracy: 0.8660	- loss: 0.4610
882/1407		1s 3ms/step	- accuracy: 0.8675	- loss: 0.4559
907/1407		1s 3ms/step	- accuracy: 0.8689	- loss: 0.4510
933/1407		1s 3ms/step	- accuracy: 0.8703	- loss: 0.4461
960/1407		1s 3ms/step	- accuracy: 0.8718	- loss: 0.4411
987/1407		1s 3ms/step	- accuracy: 0.8732	- loss: 0.4364
1013/1407		1s 3ms/step	- accuracy: 0.8744	- loss: 0.4319
1036/1407		0s 3ms/step	- accuracy: 0.8756	- loss: 0.4281
1063/1407		0s 3ms/step	- accuracy: 0.8768	- loss: 0.4238
1089/1407		0s 3ms/step	- accuracy: 0.8780	- loss: 0.4198
1117/1407		0s 3ms/step	- accuracy: 0.8792	- loss: 0.4156
1145/1407		0s 3ms/step	- accuracy: 0.8804	- loss: 0.4115
1172/1407		0s 3ms/step	- accuracy: 0.8815	- loss: 0.4077
1200/1407		0s 3ms/step	- accuracy: 0.8826	- loss: 0.4038
1229/1407		0s 2ms/step	- accuracy: 0.8837	- loss: 0.4000
1258/1407		0s 2ms/step	- accuracy: 0.8848	- loss: 0.3962
1287/1407		0s 2ms/step	- accuracy: 0.8859	- loss: 0.3926
1316/1407		0s 2ms/step	- accuracy: 0.8869	- loss: 0.3891
1347/1407		0s 2ms/step	- accuracy: 0.8879	- loss: 0.3854
1376/1407		0s 2ms/step	- accuracy: 0.8889	- loss: 0.3821
1407/1407		0s 2ms/step	- accuracy: 0.8899	- loss: 0.3787
1407/1407		5s 3ms/step	- accuracy: 0.9342	- loss: 0.2253
- val_accuracy: 0.9611 - val_loss: 0.1318				
Epoch 2/5				
1/1407		19s 14ms/step	- accuracy: 0.9688	- loss: 0.1480
23/1407		3s 2ms/step	- accuracy: 0.9810	- loss: 0.0777
45/1407		3s 2ms/step	- accuracy: 0.9798	- loss: 0.0838
65/1407		3s 2ms/step	- accuracy: 0.9782	- loss: 0.0879
71/1407		3s 3ms/step	- accuracy: 0.9779	- loss: 0.0882
80/1407		4s 3ms/step	- accuracy: 0.9776	- loss: 0.0884
100/1407		4s 3ms/step	- accuracy: 0.9766	- loss: 0.0897

126/1407		3s	3ms/step	- accuracy: 0.9749	- loss: 0.0935
154/1407		3s	3ms/step	- accuracy: 0.9739	- loss: 0.0959
181/1407		3s	3ms/step	- accuracy: 0.9731	- loss: 0.0974
209/1407		2s	2ms/step	- accuracy: 0.9725	- loss: 0.0987
236/1407		2s	2ms/step	- accuracy: 0.9720	- loss: 0.0997
264/1407		2s	2ms/step	- accuracy: 0.9715	- loss: 0.1006
293/1407		2s	2ms/step	- accuracy: 0.9712	- loss: 0.1011
320/1407		2s	2ms/step	- accuracy: 0.9709	- loss: 0.1016
348/1407		2s	2ms/step	- accuracy: 0.9707	- loss: 0.1019
376/1407		2s	2ms/step	- accuracy: 0.9706	- loss: 0.1021
404/1407		2s	2ms/step	- accuracy: 0.9704	- loss: 0.1023
432/1407		2s	2ms/step	- accuracy: 0.9703	- loss: 0.1024
460/1407		2s	2ms/step	- accuracy: 0.9702	- loss: 0.1026
487/1407		1s	2ms/step	- accuracy: 0.9701	- loss: 0.1027
514/1407		1s	2ms/step	- accuracy: 0.9700	- loss: 0.1028
541/1407		1s	2ms/step	- accuracy: 0.9699	- loss: 0.1029
569/1407		1s	2ms/step	- accuracy: 0.9699	- loss: 0.1029
598/1407		1s	2ms/step	- accuracy: 0.9698	- loss: 0.1029
628/1407		1s	2ms/step	- accuracy: 0.9698	- loss: 0.1028
658/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1027
688/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1026
718/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1025
749/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1023
780/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1022
811/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1020
842/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1019
872/1407		1s	2ms/step	- accuracy: 0.9697	- loss: 0.1018
901/1407		0s	2ms/step	- accuracy: 0.9697	- loss: 0.1016
933/1407		0s	2ms/step	- accuracy: 0.9697	- loss: 0.1015
964/1407		0s	2ms/step	- accuracy: 0.9697	- loss: 0.1013

995/1407		0s 2ms/step	- accuracy: 0.9698	- loss: 0.1011
1026/1407		0s 2ms/step	- accuracy: 0.9698	- loss: 0.1009
1056/1407		0s 2ms/step	- accuracy: 0.9699	- loss: 0.1007
1085/1407		0s 2ms/step	- accuracy: 0.9699	- loss: 0.1005
1111/1407		0s 2ms/step	- accuracy: 0.9699	- loss: 0.1003
1139/1407		0s 2ms/step	- accuracy: 0.9700	- loss: 0.1001
1168/1407		0s 2ms/step	- accuracy: 0.9700	- loss: 0.0999
1195/1407		0s 2ms/step	- accuracy: 0.9701	- loss: 0.0997
1225/1407		0s 2ms/step	- accuracy: 0.9701	- loss: 0.0995
1255/1407		0s 2ms/step	- accuracy: 0.9702	- loss: 0.0992
1285/1407		0s 2ms/step	- accuracy: 0.9702	- loss: 0.0990
1315/1407		0s 2ms/step	- accuracy: 0.9703	- loss: 0.0988
1345/1407		0s 2ms/step	- accuracy: 0.9703	- loss: 0.0985
1376/1407		0s 2ms/step	- accuracy: 0.9704	- loss: 0.0983
1406/1407		0s 2ms/step	- accuracy: 0.9705	- loss: 0.0981
1407/1407		3s 2ms/step	- accuracy: 0.9730	- loss: 0.0878
- val_accuracy: 0.9651 - val_loss: 0.1123				
Epoch 3/5				
1/1407		12s 9ms/step	- accuracy: 0.9688	- loss: 0.1168
30/1407		2s 2ms/step	- accuracy: 0.9915	- loss: 0.0478
60/1407		2s 2ms/step	- accuracy: 0.9895	- loss: 0.0531
88/1407		2s 2ms/step	- accuracy: 0.9890	- loss: 0.0526
104/1407		2s 2ms/step	- accuracy: 0.9885	- loss: 0.0531
127/1407		2s 2ms/step	- accuracy: 0.9875	- loss: 0.0551
155/1407		2s 2ms/step	- accuracy: 0.9865	- loss: 0.0567
184/1407		2s 2ms/step	- accuracy: 0.9858	- loss: 0.0578
213/1407		2s 2ms/step	- accuracy: 0.9852	- loss: 0.0586
243/1407		2s 2ms/step	- accuracy: 0.9847	- loss: 0.0591
273/1407		2s 2ms/step	- accuracy: 0.9843	- loss: 0.0596
303/1407		2s 2ms/step	- accuracy: 0.9841	- loss: 0.0599
333/1407		1s 2ms/step	- accuracy: 0.9839	- loss: 0.0602

364/1407		1s 2ms/step	accuracy: 0.9838	loss: 0.0604
395/1407		1s 2ms/step	accuracy: 0.9837	loss: 0.0605
425/1407		1s 2ms/step	accuracy: 0.9836	loss: 0.0606
454/1407		1s 2ms/step	accuracy: 0.9835	loss: 0.0608
479/1407		1s 2ms/step	accuracy: 0.9834	loss: 0.0609
499/1407		1s 2ms/step	accuracy: 0.9834	loss: 0.0610
508/1407		1s 2ms/step	accuracy: 0.9833	loss: 0.0610
513/1407		1s 2ms/step	accuracy: 0.9833	loss: 0.0611
521/1407		1s 2ms/step	accuracy: 0.9833	loss: 0.0611
540/1407		1s 2ms/step	accuracy: 0.9833	loss: 0.0611
559/1407		1s 2ms/step	accuracy: 0.9833	loss: 0.0612
582/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
604/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
623/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
644/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
664/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
684/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0612
705/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0611
725/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0610
745/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0610
764/1407		1s 2ms/step	accuracy: 0.9832	loss: 0.0610
783/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0609
809/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0609
834/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0609
861/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0608
888/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0608
912/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0607
933/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0607
951/1407		1s 2ms/step	accuracy: 0.9831	loss: 0.0606
976/1407		0s 2ms/step	accuracy: 0.9831	loss: 0.0605

1003/1407  0s 2ms/step - accuracy: 0.9831 - loss: 0.0604

1031/1407  0s 2ms/step - accuracy: 0.9831 - loss: 0.0603

1060/1407  0s 2ms/step - accuracy: 0.9831 - loss: 0.0602

1088/1407  0s 2ms/step - accuracy: 0.9831 - loss: 0.0601

1118/1407  0s 2ms/step - accuracy: 0.9832 - loss: 0.0600

1149/1407  0s 2ms/step - accuracy: 0.9832 - loss: 0.0598

1179/1407  0s 2ms/step - accuracy: 0.9832 - loss: 0.0597

1209/1407  0s 2ms/step - accuracy: 0.9832 - loss: 0.0596

1236/1407  0s 2ms/step - accuracy: 0.9833 - loss: 0.0594

1259/1407  0s 2ms/step - accuracy: 0.9833 - loss: 0.0593

1280/1407  0s 2ms/step - accuracy: 0.9833 - loss: 0.0592

1303/1407  0s 2ms/step - accuracy: 0.9833 - loss: 0.0591

1327/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0590

1347/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0589

1366/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0589

1387/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0588

1397/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0587

1402/1407  0s 2ms/step - accuracy: 0.9834 - loss: 0.0587

1407/1407  3s 2ms/step - accuracy: 0.9849 - loss: 0.0526
- val_accuracy: 0.9675 - val_loss: 0.1081
Epoch 4/5

1/1407  13s 10ms/step - accuracy: 0.9688 - loss: 0.0552

31/1407  2s 2ms/step - accuracy: 0.9903 - loss: 0.0304

61/1407  2s 2ms/step - accuracy: 0.9908 - loss: 0.0341

92/1407  2s 2ms/step - accuracy: 0.9914 - loss: 0.0336

122/1407  2s 2ms/step - accuracy: 0.9909 - loss: 0.0345

144/1407  2s 2ms/step - accuracy: 0.9905 - loss: 0.0352

170/1407  2s 2ms/step - accuracy: 0.9902 - loss: 0.0356

197/1407  2s 2ms/step - accuracy: 0.9900 - loss: 0.0358

227/1407  2s 2ms/step - accuracy: 0.9899 - loss: 0.0359

256/1407  2s 2ms/step - accuracy: 0.9898 - loss: 0.0361

271/1407		2s	2ms/step	-	accuracy: 0.9898	-	loss: 0.0362
300/1407		2s	2ms/step	-	accuracy: 0.9898	-	loss: 0.0363
330/1407		2s	2ms/step	-	accuracy: 0.9898	-	loss: 0.0365
359/1407		1s	2ms/step	-	accuracy: 0.9898	-	loss: 0.0366
388/1407		1s	2ms/step	-	accuracy: 0.9898	-	loss: 0.0367
415/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0367
446/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0368
478/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0369
508/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0370
540/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0370
573/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0370
605/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0371
637/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0371
667/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0371
697/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0371
728/1407		1s	2ms/step	-	accuracy: 0.9899	-	loss: 0.0371
758/1407		1s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0371
788/1407		1s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0371
819/1407		1s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0371
837/1407		1s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0371
849/1407		1s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0370
864/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0370
895/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0370
926/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0370
958/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0369
991/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0369
1023/1407		0s	2ms/step	-	accuracy: 0.9900	-	loss: 0.0368
1053/1407		0s	2ms/step	-	accuracy: 0.9901	-	loss: 0.0368
1084/1407		0s	2ms/step	-	accuracy: 0.9901	-	loss: 0.0367
1114/1407		0s	2ms/step	-	accuracy: 0.9901	-	loss: 0.0367

1145/1407  0s 2ms/step - accuracy: 0.9901 - loss: 0.0366

1176/1407  0s 2ms/step - accuracy: 0.9901 - loss: 0.0365

1206/1407  0s 2ms/step - accuracy: 0.9902 - loss: 0.0365

1235/1407  0s 2ms/step - accuracy: 0.9902 - loss: 0.0364

1263/1407  0s 2ms/step - accuracy: 0.9902 - loss: 0.0363

1292/1407  0s 2ms/step - accuracy: 0.9902 - loss: 0.0362

1318/1407  0s 2ms/step - accuracy: 0.9902 - loss: 0.0362

1346/1407  0s 2ms/step - accuracy: 0.9903 - loss: 0.0361

1371/1407  0s 2ms/step - accuracy: 0.9903 - loss: 0.0360

1392/1407  0s 2ms/step - accuracy: 0.9903 - loss: 0.0360

1407/1407  3s 2ms/step - accuracy: 0.9911 - loss: 0.0326
- val_accuracy: 0.9690 - val_loss: 0.1139
Epoch 5/5

1/1407  13s 9ms/step - accuracy: 1.0000 - loss: 0.0304

31/1407  2s 2ms/step - accuracy: 0.9949 - loss: 0.0187

64/1407  2s 2ms/step - accuracy: 0.9941 - loss: 0.0221

98/1407  2s 2ms/step - accuracy: 0.9944 - loss: 0.0216

132/1407  1s 2ms/step - accuracy: 0.9944 - loss: 0.0218

166/1407  1s 2ms/step - accuracy: 0.9945 - loss: 0.0217

199/1407  1s 2ms/step - accuracy: 0.9945 - loss: 0.0217

229/1407  1s 2ms/step - accuracy: 0.9945 - loss: 0.0217

260/1407  1s 2ms/step - accuracy: 0.9945 - loss: 0.0218

291/1407  1s 2ms/step - accuracy: 0.9944 - loss: 0.0220

321/1407  1s 2ms/step - accuracy: 0.9944 - loss: 0.0223

351/1407  1s 2ms/step - accuracy: 0.9943 - loss: 0.0225

382/1407  1s 2ms/step - accuracy: 0.9942 - loss: 0.0227

413/1407  1s 2ms/step - accuracy: 0.9942 - loss: 0.0228

444/1407  1s 2ms/step - accuracy: 0.9941 - loss: 0.0230

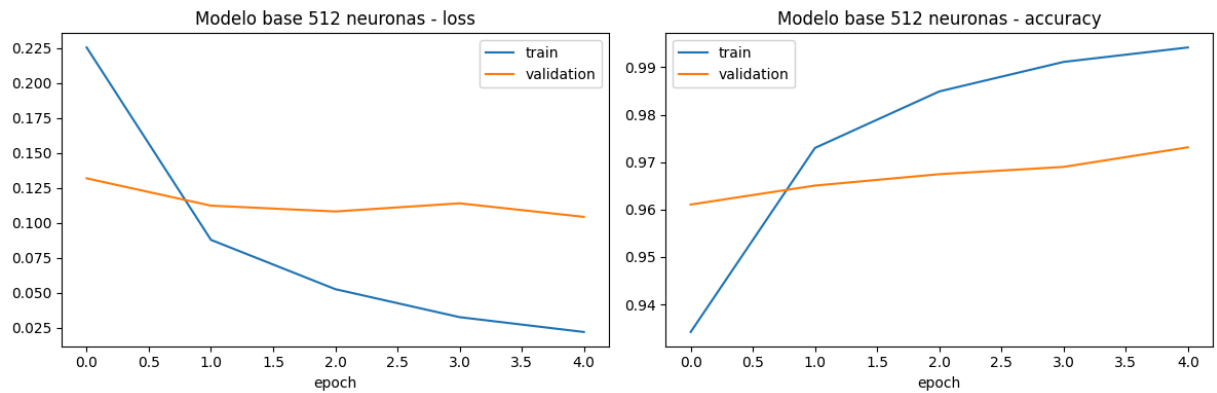
475/1407  1s 2ms/step - accuracy: 0.9940 - loss: 0.0232

506/1407  1s 2ms/step - accuracy: 0.9940 - loss: 0.0233

535/1407  1s 2ms/step - accuracy: 0.9940 - loss: 0.0234

564/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0235
594/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0235
625/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
656/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
687/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
717/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
747/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
777/1407		1s 2ms/step	accuracy: 0.9939	loss: 0.0236
807/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
837/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
867/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
897/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
926/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
956/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
986/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
1016/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
1046/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0237
1075/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
1105/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
1132/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
1162/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
1192/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0236
1222/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0235
1252/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0235
1283/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0235
1313/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0234
1343/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0234
1372/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0234
1402/1407		0s 2ms/step	accuracy: 0.9939	loss: 0.0233

1407/1407 — 3s 2ms/step — accuracy: 0.9942 — loss: 0.0219
 — val_accuracy: 0.9731 — val_loss: 0.1042



Out[15]:

	modelo	val_accuracy_f inal	val_loss_fi nal	test_accu racy	test_lo ss	tiempo_ s
0	512 neuronas, 5 epochs	0.973133	0.104241	0.9759	0.0826 02	16.5360 86

5: Impacto al variar el número de neuronas en las capas ocultas

En este ejercicio vamos a experimentar con nuestra red neuronal cambiando el número de neuronas por 512 y por otros valores. Para ello, utiliza la red neuronal de la pregunta 3, y su capa oculta cambia el número de neuronas:

- **216 neuronas en la capa oculta
- **1024 neuronas en la capa oculta

y entrena la red en ambos casos.

```
In [16]: hidden_results = []

reset_seeds()
tf.keras.backend.clear_session()
model_216 = create_mlp(hidden_units=216)
history_216, elapsed_216, test_loss_216, test_acc_216 = train_model(
    model_216, epochs=5, batch_size=32, validation_split=0.25
)
hidden_results.append({
    "neuronas": 216,
    "params": model_216.count_params(),
    "val_accuracy": history_216.history["val_accuracy"][-1],
    "test_accuracy": test_acc_216,
    "val_loss": history_216.history["val_loss"][-1],
    "tiempo_s": elapsed_216,
})

pd.DataFrame(hidden_results)
```

Out[16]:

	neuronas	params	val_accuracy	test_accuracy	val_loss	tiempo_s
--	----------	--------	--------------	---------------	----------	----------

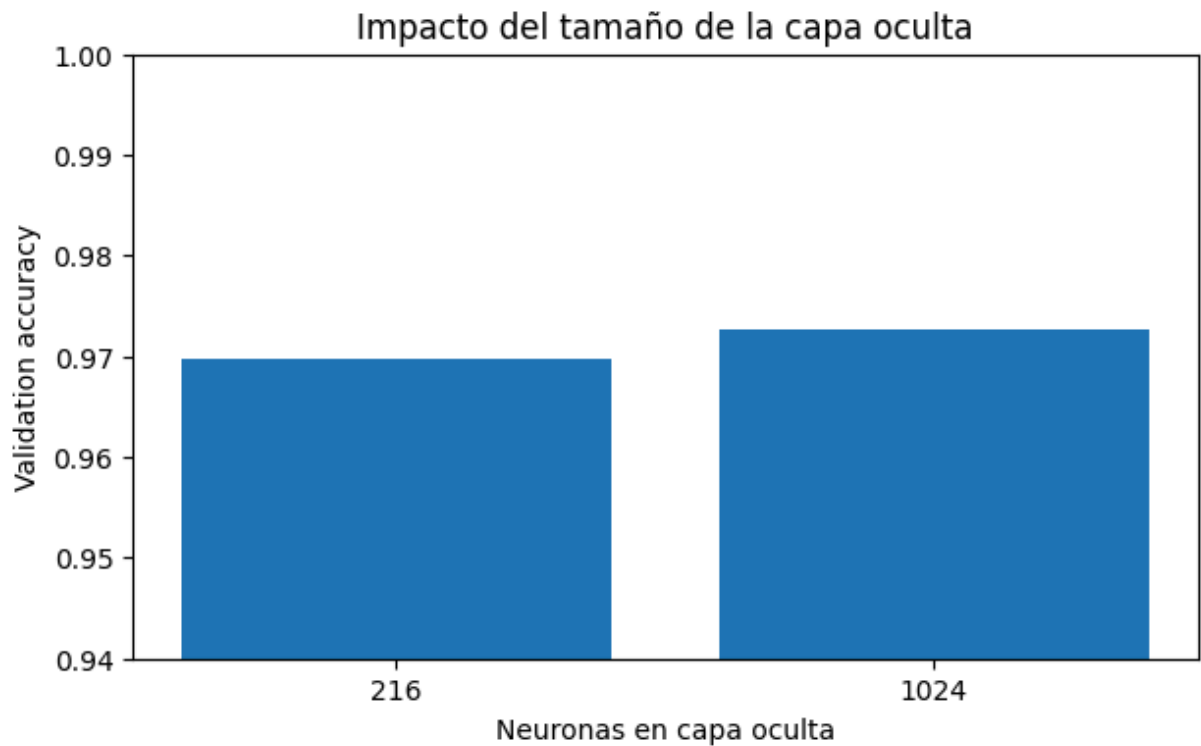
0	216	171730	0.969667	0.9744	0.1073	8.96187
---	-----	--------	----------	--------	--------	---------

```
In [17]: reset_seeds()
tf.keras.backend.clear_session()
model_1024 = create_mlp(hidden_units=1024)
history_1024, elapsed_1024, test_loss_1024, test_acc_1024 = train_model(
    model_1024, epochs=5, batch_size=32, validation_split=0.25
)
hidden_results.append({
    "neuronas": 1024,
    "params": model_1024.count_params(),
    "val_accuracy": history_1024.history["val_accuracy"][-1],
    "test_accuracy": test_acc_1024,
    "val_loss": history_1024.history["val_loss"][-1],
    "tiempo_s": elapsed_1024,
})

hidden_results_df = pd.DataFrame(hidden_results)
display(hidden_results_df)

fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(hidden_results_df["neuronas"].astype(str),
hidden_results_df["val_accuracy"])
ax.set_ylim(0.94, 1.0)
ax.set_xlabel("Neuronas en capa oculta")
ax.set_ylabel("Validation accuracy")
ax.set_title("Impacto del tamaño de la capa oculta")
plt.show()
```

	neuronas	params	val_accuracy	test_accuracy	val_loss	tiempo_s
0	216	171730	0.969667	0.9744	0.107300	8.961870
1	1024	814090	0.972600	0.9764	0.105779	23.335651



Pregunta 5.1 (0.5 puntos): ¿Cual es el impacto que tiene la red neuronal?

Respuesta a la pregunta 5.1:

Al aumentar el número de neuronas crece la capacidad del modelo y también el número de parámetros y el tiempo de entrenamiento. Con pocas neuronas el modelo puede quedarse algo más limitado, mientras que con 1024 neuronas suele mejorar algo la capacidad de ajuste. Aun así, esa mejora no siempre compensa el coste adicional y puede aumentar el riesgo de sobreajuste. En MNIST, una capa densa de 512 neuronas ya ofrece un rendimiento alto, por lo que ampliar mucho la capa oculta tiene un beneficio moderado.

6: Número de neuronas de la capa de salida

Considerad la capa final, la de salida de la red neuronal de la pregunta 3.

Pregunta 6.1 (0.25 puntos): ¿Por qué son 10 las neuronas de la última capa?

Pregunta 6.2 (0.25 puntos): ¿Qué pasaría si tuvieras una cantidad diferente a 10?

Por ejemplo, intenta entrenar la red con 5, para ello utiliza la red neuronal de la pregunta 1 y cambia a 5 el número de neuronas en la última capa.

```
In [18]: reset_seeds()
tf.keras.backend.clear_session()
model_5_outputs = create_mlp(hidden_units=512, output_units=5)
model_5_outputs.compile(optimizer="adam", loss="categorical_crossentropy",
metrics=["accuracy"])

try:
    model_5_outputs.fit(X_train, y_train, epochs=1, batch_size=128,
```

```
validation_split=0.25, verbose=0)
except Exception as exc:
    print("No se puede entrenar correctamente con 5 neuronas de salida y etiquetas one-hot de 10 clases.")
    print("Tipo de error:", type(exc).__name__)
    print("Mensaje:", str(exc).split("\n")[0])
```

No se puede entrenar correctamente con 5 neuronas de salida y etiquetas one-hot de 10 clases.

Tipo de error: ValueError

Mensaje: Arguments `target` and `output` must have the same shape. Received: target.shape=(None, 10), output.shape=(None, 5)

Respuesta a la pregunta 6.1:

La última capa tiene 10 neuronas porque MNIST tiene 10 clases posibles: los dígitos del 0 al 9. Cada neurona de salida representa la probabilidad estimada de una clase. Con `softmax`, la red devuelve una distribución de probabilidad sobre esas 10 clases y se elige la clase con mayor probabilidad.

Respuesta a la pregunta 6.2:

Si la capa final tuviera una cantidad distinta de 10 neuronas, la salida del modelo no coincidiría con el problema. Con 5 neuronas no se pueden representar las 10 clases y, además, aparece una incompatibilidad de dimensiones al comparar la salida `(N, 5)` con las etiquetas one-hot `(N, 10)`. Si hubiera más de 10 neuronas, existirían clases de salida sin significado en el dataset, lo que también sería un diseño incorrecto.

7: Aumento de epoch y su efecto en la red neuronal

En este ejercicio vamos a ver el impacto de aumentar los epoch en el entrenamiento.

Usando la red neuronal de la pregunta 3:

Pregunta 7.1 (0.25 puntos)

- Intentad 15 epoch para su entrenamiento, probablemente obtendras un modelo con una pérdida mucho mejor que el que tiene 5.

Pregunta 7.2 (0.25 puntos)

- Intenta ahora con 30 epoch para su entrenamiento.

Pregunta 7.3 (0.25 puntos)

- ¿Qué está pasando en la pregunta anterior? Explica tu respuesta y da el nombre de este efecto si lo conoces.

```
In [19]: reset_seeds()
tf.keras.backend.clear_session()
model_15 = create_mlp(hidden_units=512)
history_15, time_15, test_loss_15, test_acc_15 = train_model(
    model_15, epochs=15, batch_size=128, validation_split=0.25
```



```
)
plot_history(history_15, "Modelo base 15 epochs")

pd.DataFrame([
    "epochs": 15,
    "val_accuracy_final": history_15.history["val_accuracy"][-1],
    "val_loss_final": history_15.history["val_loss"][-1],
    "test_accuracy": test_acc_15,
    "test_loss": test_loss_15,
    "tiempo_s": time_15,
])
```



```
Out[19]:
```

	epochs	val_accuracy_final	val_loss_final	test_accuracy	test_loss	tiempo_s
0	15	0.969333	0.130926	0.9748	0.107547	14.442061

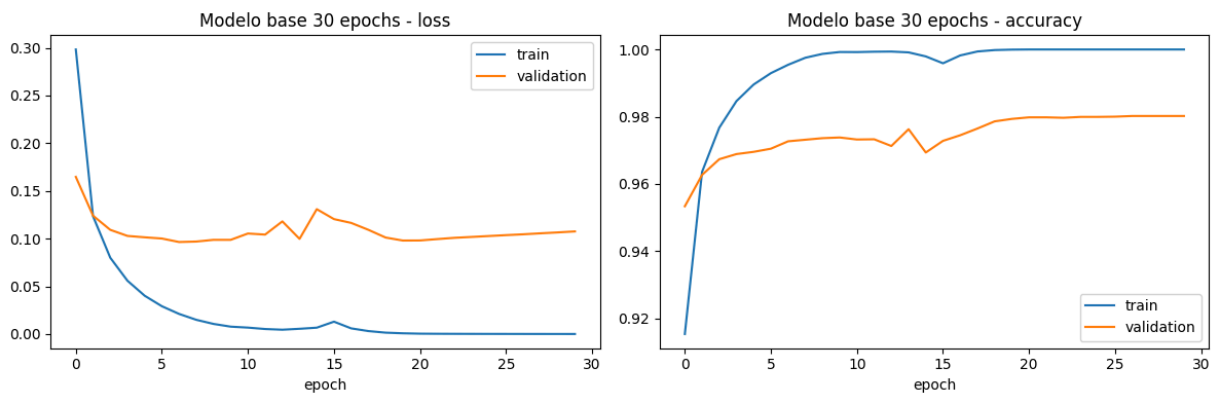
```
In [20]: reset_seeds()
tf.keras.backend.clear_session()
model_30 = create_mlp(hidden_units=512)
history_30, time_30, test_loss_30, test_acc_30 = train_model(
    model_30, epochs=30, batch_size=128, validation_split=0.25
)
plot_history(history_30, "Modelo base 30 epochs")

epochs_results_df = pd.DataFrame([
    {
        "epochs": 5,
        "val_accuracy_final": history_base.history["val_accuracy"][-1],
        "val_loss_final": history_base.history["val_loss"][-1],
        "test_accuracy": test_acc_base,
        "test_loss": test_loss_base,
        "tiempo_s": time_base,
    },
    {
        "epochs": 15,
        "val_accuracy_final": history_15.history["val_accuracy"][-1],
        "val_loss_final": history_15.history["val_loss"][-1],
        "test_accuracy": test_acc_15,
        "test_loss": test_loss_15,
        "tiempo_s": time_15,
    },
    {
        "epochs": 30,
        "val_accuracy_final": history_30.history["val_accuracy"][-1],
        "val_loss_final": history_30.history["val_loss"][-1],
        "test_accuracy": test_acc_30,
        "test_loss": test_loss_30,
    },
])
```

```

        "tiempo_s": time_30,
    },
])
display(epochs_results_df)

```



	epochs	val_accuracy_final	val_loss_final	test_accuracy	test_loss	tiempo_s
0	5	0.973133	0.104241	0.9759	0.082602	16.536086
1	15	0.969333	0.130926	0.9748	0.107547	14.442061
2	30	0.980200	0.107605	0.9830	0.087178	29.209817

Respuesta a las preguntas 7.1 y 7.2:

Al aumentar los epochs, el modelo dispone de más pasadas sobre los datos y normalmente mejora la pérdida de entrenamiento. Aun así, llega un momento en el que la mejora en validación se estabiliza e incluso puede empeorar si el modelo empieza a memorizar demasiado el conjunto de entrenamiento. Por eso no conviene elegir un número alto de epochs sin revisar las curvas de `loss` y `accuracy`. En este caso, entrenar durante más tiempo ayuda al principio, pero a partir de cierto punto es más razonable usar `EarlyStopping`.

8: Early stop

En el ejercicio anterior, cuando entrenabas con epoch extras, tenías un problema en el que tu pérdida podía cambiar. Puede que te haya llevado un poco de tiempo esperar a que el entrenamiento lo hiciera, y puede que hayas pensado "¿no estaría bien si pudiera parar el entrenamiento cuando alcance un valor deseado?", es decir, una precisión del 85% podría ser suficiente para ti, y si alcanzas eso después de 3 epoch, ¿por qué sentarte a esperar a que termine muchas más épocas? Como cualquier otro programa existen formas de parar la ejecución

A partir del código de ejemplo, hacer una nueva función que tenga en cuenta la pérdida (loss) y que pueda parar el código para evitar que ocurra el efecto secundario que vimos en el ejercicio 5.

```

In [21]: ### Ejemplo de código

class myCallback(tf.keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs={}):

```

```

        if(logs.get('accuracy') > 0.85):
            print("\nAlcanzado el 85% de precisión, se cancela el
entrenamiento!!")
            self.model.stop_training = True

```

****Pregunta 8.1. *(0.75 puntos)**:** Consulta la documentación de Keras y aprende cómo podemos utilizar Early stop en nuestros modelos.

```

In [22]: reset_seeds()
tf.keras.backend.clear_session()

early_stop = keras.callbacks.EarlyStopping(
    monitor="val_loss",
    patience=3,
    min_delta=1e-4,
    restore_best_weights=True,
    verbose=1,
)

model_early = create_mlp(hidden_units=512)
history_early, time_early, test_loss_early, test_acc_early = train_model(
    model_early,
    epochs=50,
    batch_size=128,
    validation_split=0.25,
    callbacks=[early_stop],
    verbose=1,
)
plot_history(history_early, "Modelo con EarlyStopping")

print(f"Entrenamiento detenido tras {len(history_early.history['loss'])}
epochs.")
print(f"Mejor val_loss: {min(history_early.history['val_loss']):.4f}")
print(f"Test accuracy tras restaurar los mejores pesos:
{test_acc_early:.4f}")

```

```

Epoch 1/50
 1/352 _____ 2:02 349ms/step - accuracy: 0.0703 - loss:
2.3473

18/352 _____ 0s 3ms/step - accuracy: 0.4952 - loss: 1.6506

39/352 _____ 0s 3ms/step - accuracy: 0.6297 - loss: 1.2657

58/352 _____ 0s 3ms/step - accuracy: 0.6872 - loss: 1.0822

77/352 _____ 0s 3ms/step - accuracy: 0.7224 - loss: 0.9644

96/352 _____ 0s 3ms/step - accuracy: 0.7469 - loss: 0.8808

113/352 _____ 0s 3ms/step - accuracy: 0.7638 - loss: 0.8234

130/352 _____ 0s 3ms/step - accuracy: 0.7773 - loss: 0.7774

147/352 _____ 0s 3ms/step - accuracy: 0.7884 - loss: 0.7393

163/352 _____ 0s 3ms/step - accuracy: 0.7973 - loss: 0.7088

181/352 _____ 0s 3ms/step - accuracy: 0.8058 - loss: 0.6792

```

186/352  0s 3ms/step - accuracy: 0.8080 - loss: 0.6717

190/352  0s 3ms/step - accuracy: 0.8097 - loss: 0.6659

197/352  0s 4ms/step - accuracy: 0.8125 - loss: 0.6562

211/352  0s 4ms/step - accuracy: 0.8177 - loss: 0.6380

226/352  0s 4ms/step - accuracy: 0.8229 - loss: 0.6203

242/352  0s 4ms/step - accuracy: 0.8279 - loss: 0.6031

257/352  0s 4ms/step - accuracy: 0.8322 - loss: 0.5883

271/352  0s 4ms/step - accuracy: 0.8359 - loss: 0.5755

287/352  0s 4ms/step - accuracy: 0.8398 - loss: 0.5619

300/352  0s 4ms/step - accuracy: 0.8428 - loss: 0.5516

303/352  0s 4ms/step - accuracy: 0.8435 - loss: 0.5493

306/352  0s 4ms/step - accuracy: 0.8441 - loss: 0.5470

312/352  0s 4ms/step - accuracy: 0.8454 - loss: 0.5426

331/352  0s 4ms/step - accuracy: 0.8492 - loss: 0.5292

352/352  0s 4ms/step - accuracy: 0.8531 - loss: 0.5157

352/352  2s 5ms/step - accuracy: 0.9153 - loss: 0.2983 -
val_accuracy: 0.9533 - val_loss: 0.1648
Epoch 2/50

1/352  3s 11ms/step - accuracy: 0.9766 - loss: 0.1097

22/352  0s 2ms/step - accuracy: 0.9619 - loss: 0.1265

42/352  0s 3ms/step - accuracy: 0.9603 - loss: 0.1322

61/352  0s 3ms/step - accuracy: 0.9593 - loss: 0.1354

82/352  0s 3ms/step - accuracy: 0.9585 - loss: 0.1374

102/352  0s 3ms/step - accuracy: 0.9581 - loss: 0.1383

123/352  0s 3ms/step - accuracy: 0.9579 - loss: 0.1390

144/352  0s 3ms/step - accuracy: 0.9578 - loss: 0.1395

165/352  0s 3ms/step - accuracy: 0.9577 - loss: 0.1398

189/352  0s 2ms/step - accuracy: 0.9578 - loss: 0.1397

209/352  0s 2ms/step - accuracy: 0.9579 - loss: 0.1395

232/352  0s 2ms/step - accuracy: 0.9581 - loss: 0.1391

256/352  0s 2ms/step - accuracy: 0.9583 - loss: 0.1385

280/352  0s 2ms/step - accuracy: 0.9586 - loss: 0.1379

303/352  0s 2ms/step - accuracy: 0.9589 - loss: 0.1371

316/352  0s 2ms/step - accuracy: 0.9590 - loss: 0.1366

338/352  0s 2ms/step - accuracy: 0.9593 - loss: 0.1358

352/352  1s 3ms/step - accuracy: 0.9636 - loss: 0.1235 -
val_accuracy: 0.9627 - val_loss: 0.1241
Epoch 3/50

1/352  3s 10ms/step - accuracy: 0.9844 - loss: 0.0598

25/352  0s 2ms/step - accuracy: 0.9781 - loss: 0.0732

50/352  0s 2ms/step - accuracy: 0.9768 - loss: 0.0796

75/352  0s 2ms/step - accuracy: 0.9759 - loss: 0.0828

100/352  0s 2ms/step - accuracy: 0.9755 - loss: 0.0842

125/352  0s 2ms/step - accuracy: 0.9752 - loss: 0.0852

148/352  0s 2ms/step - accuracy: 0.9751 - loss: 0.0859

172/352  0s 2ms/step - accuracy: 0.9750 - loss: 0.0863

190/352  0s 2ms/step - accuracy: 0.9750 - loss: 0.0864

212/352  0s 2ms/step - accuracy: 0.9750 - loss: 0.0865

236/352  0s 2ms/step - accuracy: 0.9750 - loss: 0.0865

261/352  0s 2ms/step - accuracy: 0.9751 - loss: 0.0863

286/352  0s 2ms/step - accuracy: 0.9751 - loss: 0.0861

310/352  0s 2ms/step - accuracy: 0.9752 - loss: 0.0857

335/352  0s 2ms/step - accuracy: 0.9753 - loss: 0.0853

352/352  1s 2ms/step - accuracy: 0.9767 - loss: 0.0801 -
val_accuracy: 0.9673 - val_loss: 0.1093
Epoch 4/50

1/352  3s 9ms/step - accuracy: 0.9922 - loss: 0.0428

25/352  0s 2ms/step - accuracy: 0.9879 - loss: 0.0507

48/352  0s 2ms/step - accuracy: 0.9858 - loss: 0.0551

69/352  0s 2ms/step - accuracy: 0.9847 - loss: 0.0574

93/352  0s 2ms/step - accuracy: 0.9841 - loss: 0.0586

117/352  0s 2ms/step - accuracy: 0.9839 - loss: 0.0591

142/352  0s 2ms/step - accuracy: 0.9838 - loss: 0.0596

167/352  0s 2ms/step - accuracy: 0.9837 - loss: 0.0599

192/352  0s 2ms/step - accuracy: 0.9837 - loss: 0.0600

217/352  0s 2ms/step - accuracy: 0.9837 - loss: 0.0601

242/352  0s 2ms/step - accuracy: 0.9837 - loss: 0.0602

267/352  0s 2ms/step - accuracy: 0.9837 - loss: 0.0600

292/352  0s 2ms/step - accuracy: 0.9838 - loss: 0.0598

317/352  0s 2ms/step - accuracy: 0.9838 - loss: 0.0596

342/352  0s 2ms/step - accuracy: 0.9839 - loss: 0.0593

352/352  1s 2ms/step - accuracy: 0.9846 - loss: 0.0559 -
val_accuracy: 0.9689 - val_loss: 0.1029
Epoch 5/50

1/352  3s 10ms/step - accuracy: 0.9922 - loss: 0.0333

24/352  0s 2ms/step - accuracy: 0.9927 - loss: 0.0381

47/352  0s 2ms/step - accuracy: 0.9909 - loss: 0.0408

71/352  0s 2ms/step - accuracy: 0.9900 - loss: 0.0421

95/352  0s 2ms/step - accuracy: 0.9896 - loss: 0.0427

116/352  0s 2ms/step - accuracy: 0.9894 - loss: 0.0428

138/352  0s 2ms/step - accuracy: 0.9893 - loss: 0.0430

160/352  0s 2ms/step - accuracy: 0.9892 - loss: 0.0432

182/352  0s 2ms/step - accuracy: 0.9892 - loss: 0.0432

204/352  0s 2ms/step - accuracy: 0.9891 - loss: 0.0433

225/352  0s 2ms/step - accuracy: 0.9890 - loss: 0.0434

247/352  0s 2ms/step - accuracy: 0.9890 - loss: 0.0434

268/352  0s 2ms/step - accuracy: 0.9890 - loss: 0.0433

290/352  0s 2ms/step - accuracy: 0.9890 - loss: 0.0432

313/352  0s 2ms/step - accuracy: 0.9891 - loss: 0.0430

334/352  0s 2ms/step - accuracy: 0.9891 - loss: 0.0428

352/352  1s 3ms/step - accuracy: 0.9896 - loss: 0.0402 -
val_accuracy: 0.9695 - val_loss: 0.1014
Epoch 6/50

1/352  3s 11ms/step - accuracy: 0.9922 - loss: 0.0229

23/352  0s 2ms/step - accuracy: 0.9934 - loss: 0.0287

45/352  0s 2ms/step - accuracy: 0.9928 - loss: 0.0304

68/352  0s 2ms/step - accuracy: 0.9925 - loss: 0.0310

90/352  0s 2ms/step - accuracy: 0.9924 - loss: 0.0314

113/352  0s 2ms/step - accuracy: 0.9924 - loss: 0.0314

134/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0316

156/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0316

178/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0317

201/352  0s 2ms/step - accuracy: 0.9922 - loss: 0.0317

224/352  0s 2ms/step - accuracy: 0.9922 - loss: 0.0318

247/352  0s 2ms/step - accuracy: 0.9922 - loss: 0.0318

270/352  0s 2ms/step - accuracy: 0.9922 - loss: 0.0317

293/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0316

316/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0314

339/352  0s 2ms/step - accuracy: 0.9923 - loss: 0.0312

352/352  1s 3ms/step - accuracy: 0.9930 - loss: 0.0292 -
val_accuracy: 0.9705 - val_loss: 0.1002

Epoch 7/50

1/352  3s 10ms/step - accuracy: 1.0000 - loss: 0.0144

25/352  0s 2ms/step - accuracy: 0.9955 - loss: 0.0215

50/352  0s 2ms/step - accuracy: 0.9948 - loss: 0.0229

75/352  0s 2ms/step - accuracy: 0.9946 - loss: 0.0232

99/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0234

124/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0234

149/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0234

174/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0235

197/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0235

222/352  0s 2ms/step - accuracy: 0.9945 - loss: 0.0235

247/352  0s 2ms/step - accuracy: 0.9946 - loss: 0.0234

270/352  0s 2ms/step - accuracy: 0.9946 - loss: 0.0233

293/352  0s 2ms/step - accuracy: 0.9946 - loss: 0.0232

317/352  0s 2ms/step - accuracy: 0.9947 - loss: 0.0231

339/352  0s 2ms/step - accuracy: 0.9947 - loss: 0.0229

352/352  1s 2ms/step - accuracy: 0.9954 - loss: 0.0211 -
val_accuracy: 0.9727 - val_loss: 0.0965

Epoch 8/50

1/352  3s 10ms/step - accuracy: 1.0000 - loss: 0.0092

24/352  0s 2ms/step - accuracy: 0.9961 - loss: 0.0147

48/352  0s 2ms/step - accuracy: 0.9963 - loss: 0.0161

71/352  0s 2ms/step - accuracy: 0.9964 - loss: 0.0164

94/352  0s 2ms/step - accuracy: 0.9965 - loss: 0.0166

118/352  0s 2ms/step - accuracy: 0.9966 - loss: 0.0166

142/352  0s 2ms/step - accuracy: 0.9966 - loss: 0.0166

165/352  0s 2ms/step - accuracy: 0.9966 - loss: 0.0167

189/352  0s 2ms/step - accuracy: 0.9966 - loss: 0.0167

212/352  0s 2ms/step - accuracy: 0.9967 - loss: 0.0167

235/352  0s 2ms/step - accuracy: 0.9967 - loss: 0.0166

257/352  0s 2ms/step - accuracy: 0.9967 - loss: 0.0166

281/352  0s 2ms/step - accuracy: 0.9968 - loss: 0.0165

305/352  0s 2ms/step - accuracy: 0.9968 - loss: 0.0164

329/352  0s 2ms/step - accuracy: 0.9969 - loss: 0.0163

352/352  0s 2ms/step - accuracy: 0.9969 - loss: 0.0162

352/352  1s 3ms/step - accuracy: 0.9975 - loss: 0.0149 - val_accuracy: 0.9731 - val_loss: 0.0970

Epoch 9/50

1/352  3s 10ms/step - accuracy: 1.0000 - loss: 0.0056

24/352  0s 2ms/step - accuracy: 0.9990 - loss: 0.0092

47/352  0s 2ms/step - accuracy: 0.9989 - loss: 0.0102

70/352  0s 2ms/step - accuracy: 0.9990 - loss: 0.0105

93/352  0s 2ms/step - accuracy: 0.9989 - loss: 0.0107

117/352  0s 2ms/step - accuracy: 0.9988 - loss: 0.0108

141/352  0s 2ms/step - accuracy: 0.9987 - loss: 0.0110

165/352  0s 2ms/step - accuracy: 0.9986 - loss: 0.0111

189/352  0s 2ms/step - accuracy: 0.9986 - loss: 0.0112

213/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0113

237/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0113

260/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0113

283/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0113

303/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0113

325/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0112

347/352  0s 2ms/step - accuracy: 0.9985 - loss: 0.0112

352/352  1s 3ms/step - accuracy: 0.9987 - loss: 0.0106 -
val_accuracy: 0.9736 - val_loss: 0.0988
Epoch 10/50

1/352  3s 10ms/step - accuracy: 1.0000 - loss: 0.0051

23/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0069

45/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0073

67/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0075

89/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0076

110/352  0s 2ms/step - accuracy: 0.9995 - loss: 0.0076

132/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0077

154/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0077

176/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0078

198/352  0s 2ms/step - accuracy: 0.9994 - loss: 0.0079

220/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0079

242/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0080

264/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0080

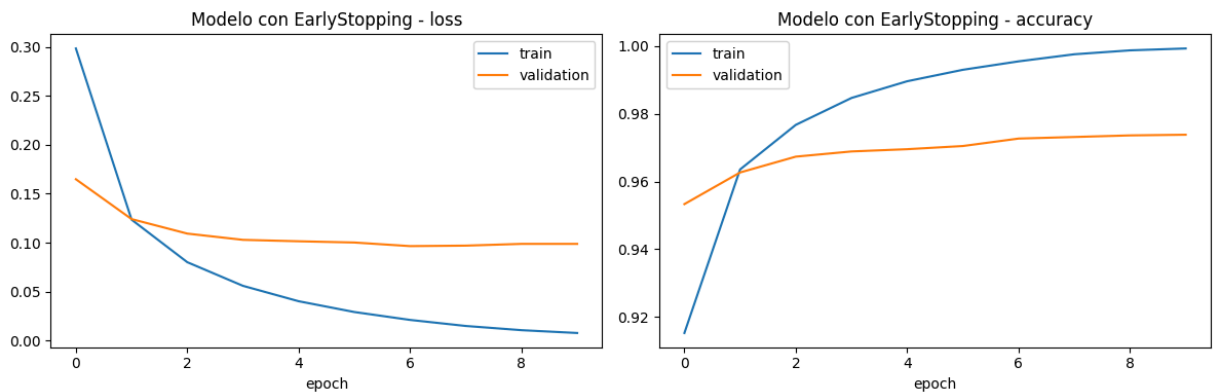
286/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0080

308/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0080

326/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0080

348/352  0s 2ms/step - accuracy: 0.9993 - loss: 0.0079

352/352  1s 3ms/step - accuracy: 0.9992 - loss: 0.0078 -
val_accuracy: 0.9738 - val_loss: 0.0988
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 7.



Entrenamiento detenido tras 10 epochs.

Mejor val_loss: 0.0965

Test accuracy tras restaurar los mejores pesos: 0.9783

9. Unidades de activación

En este ejercicio, vamos a evaluar la importancia de utilizar las unidades de activación adecuadas. Como hemos visto en clase, funciones de activación como sigmoid han dejado de utilizarse en favor de otras unidades como ReLU.

****Pregunta 9.1 *(0.75 puntos)**:** Utilizando la red realizada en el ejercicio 3, escribir un breve análisis comparando la utilización de unidades sigmoid y ReLU (por ejemplo, se pueden comentar aspectos como velocidad de convergencia, métricas obtenidas...). Explicar por qué pueden darse estas diferencias. Opcionalmente, comparar con otras activaciones disponibles en Keras.

Pista: Usando redes más grandes se hace más sencillo apreciar las diferencias. Es mejor utilizar al menos 3 o 4 capas densas.

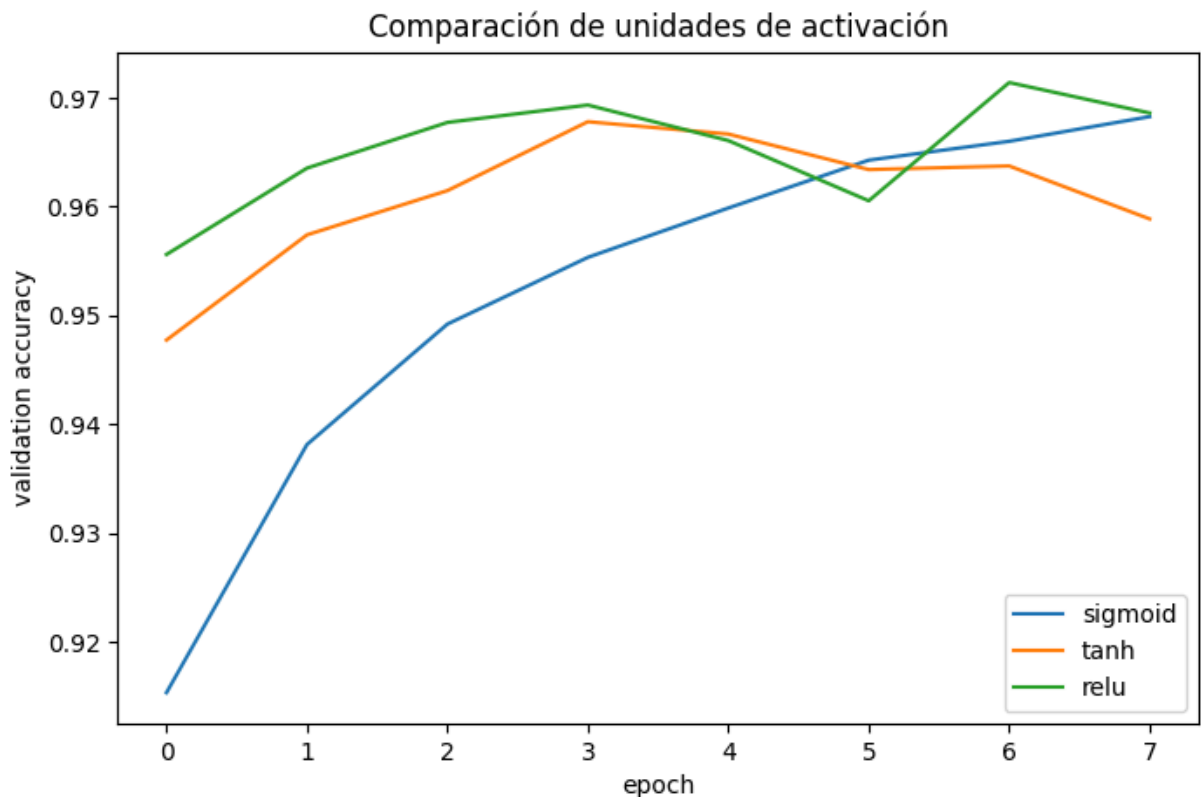
```
In [23]: activation_results = []
activation_histories = {}

for activation in ["sigmoid", "tanh", "relu"]:
    reset_seeds()
    tf.keras.backend.clear_session()
    model_act = create_mlp(
        activation=activation,
        hidden_layers=[512, 256, 128],
        initializer="glorot_uniform",
    )
    history_act, elapsed_act, test_loss_act, test_acc_act = train_model(
        model_act, epochs=8, batch_size=128, validation_split=0.25
    )
    activation_histories[activation] = history_act
    activation_results.append({
        "activación": activation,
        "val_accuracy": history_act.history["val_accuracy"][-1],
        "val_loss": history_act.history["val_loss"][-1],
        "test_accuracy": test_acc_act,
        "tiempo_s": elapsed_act,
    })

activation_results_df = pd.DataFrame(activation_results)
display(activation_results_df)
```

```
plt.figure(figsize=(8, 5))
for activation, history in activation_histories.items():
    plt.plot(history.history["val_accuracy"], label=activation)
plt.xlabel("epoch")
plt.ylabel("validation accuracy")
plt.title("Comparación de unidades de activación")
plt.legend()
plt.show()
```

	activación	val_accuracy	val_loss	test_accuracy	tiempo_s
0	sigmoid	0.968267	0.109961	0.9705	12.472741
1	tanh	0.958867	0.148986	0.9615	11.922980
2	relu	0.968600	0.128665	0.9696	11.281216



Análisis de la pregunta 9.1:

En una red con varias capas densas, **ReLU** suele converger más rápido que **sigmoid**, porque mantiene gradientes útiles para los valores positivos y reduce el problema de saturación. **sigmoid** comprime las activaciones entre 0 y 1; cuando las neuronas se saturan, los gradientes son muy pequeños y el aprendizaje avanza peor. **tanh** suele comportarse mejor que **sigmoid** al estar centrada en cero, pero también puede saturarse. En los resultados comparativos se aprecia que **ReLU** alcanza antes una mayor **val_accuracy**, por lo que es una elección más adecuada para las capas ocultas de este MLP.

10. Inicialización de parámetros

En este ejercicio, vamos a evaluar la importancia de una correcta inicialización de parámetros en una red neuronal.

****Pregunta 10.1 *(0.75 puntos)**:** Partiendo de una red similar a la del ejercicio anterior (usando ya ReLUs), comentar las diferencias que se aprecian en el entrenamiento al utilizar distintas estrategias de inicialización de parámetros. Para ello, inicializar todas las capas con las siguientes estrategias, disponibles en Keras, y analizar sus diferencias:

- Inicialización con ceros.
- Inicialización con una variable aleatoria normal.
- Inicialización con los valores por defecto de Keras para una capa Dense (estrategia *glorot uniform*)

```
In [24]: initializer_results = []
initializer_histories = {}

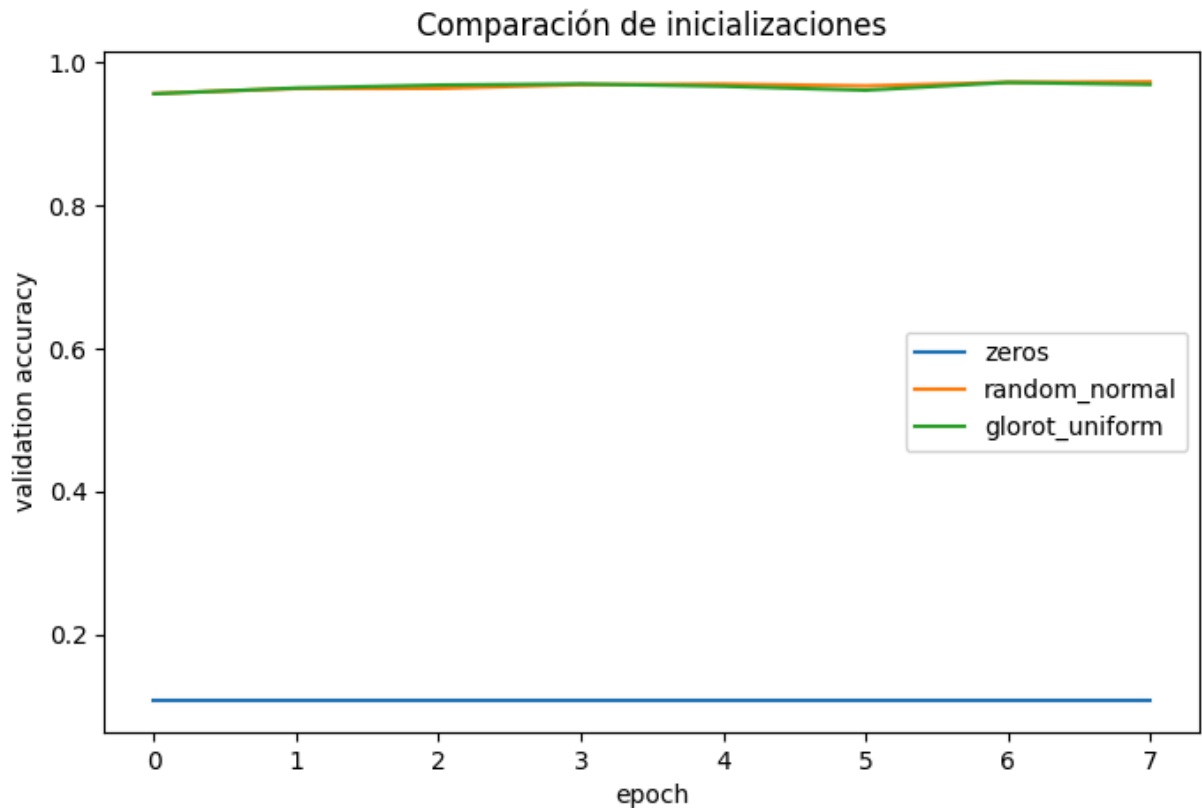
initializers_to_test = {
    "zeros": keras.initializers.Zeros(),
    "random_normal": keras.initializers.RandomNormal(mean=0.0, stddev=0.05,
seed=SEED),
    "glorot_uniform": "glorot_uniform",
}

for name, initializer in initializers_to_test.items():
    reset_seeds()
    tf.keras.backend.clear_session()
    model_init = create_mlp(
        activation="relu",
        hidden_layers=[512, 256, 128],
        initializer=initializer,
    )
    history_init, elapsed_init, test_loss_init, test_acc_init =
train_model(
        model_init, epochs=8, batch_size=128, validation_split=0.25
    )
    initializer_histories[name] = history_init
    initializer_results.append({
        "inicialización": name,
        "val_accuracy": history_init.history["val_accuracy"][-1],
        "val_loss": history_init.history["val_loss"][-1],
        "test_accuracy": test_acc_init,
        "tiempo_s": elapsed_init,
    })

initializer_results_df = pd.DataFrame(initializer_results)
display(initializer_results_df)

plt.figure(figsize=(8, 5))
for name, history in initializer_histories.items():
    plt.plot(history.history["val_accuracy"], label=name)
plt.xlabel("epoch")
plt.ylabel("validation accuracy")
plt.title("Comparación de inicializaciones")
plt.legend()
plt.show()
```

	inicialización	val_accuracy	val_loss	test_accuracy	tiempo_s
0	zeros	0.107600	2.301830	0.1135	11.535613
1	random_normal	0.972267	0.128155	0.9765	11.700772
2	glorot_uniform	0.968600	0.128665	0.9696	11.792779



Análisis de la pregunta 10.1:

La inicialización con ceros no es adecuada para redes densas profundas, porque todas las neuronas de una misma capa empiezan iguales y reciben actualizaciones muy parecidas. En la práctica, esto impide que aprendan representaciones distintas. La inicialización aleatoria normal rompe esa simetría y permite aprender, aunque su escala puede hacer que los gradientes sean menos estables. La inicialización por defecto de Keras para una capa `Dense`, `glorot_uniform`, ofrece un punto de partida más equilibrado y suele entrenar de forma más estable. Para redes con `ReLU` también sería razonable probar `he_uniform`, pero en esta comparación `glorot_uniform` ya funciona claramente mejor que ceros.

11. Optimizadores

****Problema 11.1 *(0.75 puntos)**:** Partiendo de una red similar a la del ejercicio anterior (utilizando la mejor estrategia de inicialización observada), comparar y analizar las diferencias que se observan al entrenar con varios de los optimizadores vistos en clase, incluyendo SGD como optimizador básico (se puede explorar el espacio de hiperparámetros de cada optimizador, aunque para optimizadores más avanzados del estilo de RMSprop es buena idea dejar los valores por defecto provistos por Keras).

```

In [25]: optimizer_results = []
optimizer_histories = {}

optimizers_to_test = {
    "SGD lr=0.01": keras.optimizers.SGD(learning_rate=0.01),
    "SGD momentum=0.9": keras.optimizers.SGD(learning_rate=0.01,
momentum=0.9),
    "RMSprop": keras.optimizers.RMSprop(),
    "Adam": keras.optimizers.Adam(),
}

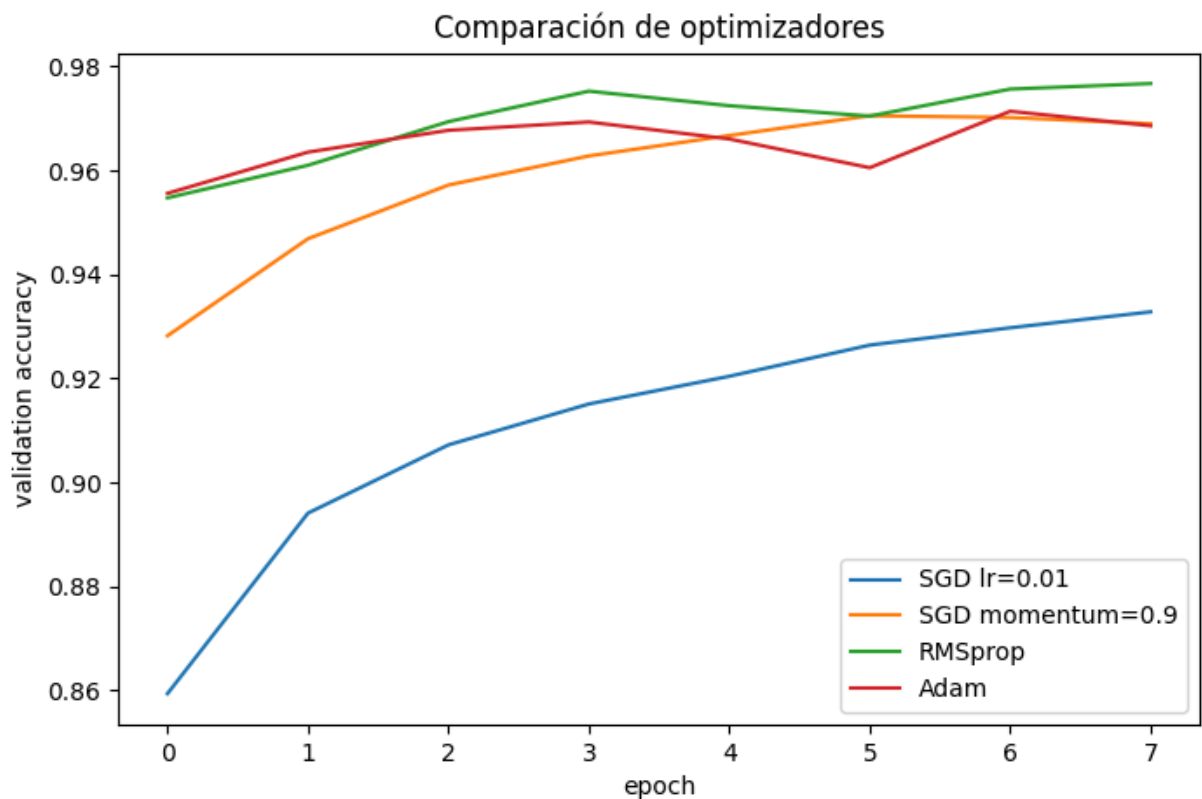
for name, optimizer in optimizers_to_test.items():
    reset_seeds()
    tf.keras.backend.clear_session()
    model_opt = create_mlp(
        activation="relu",
        hidden_layers=[512, 256, 128],
        initializer="glorot_uniform",
    )
    history_opt, elapsed_opt, test_loss_opt, test_acc_opt = train_model(
        model_opt,
        epochs=8,
        batch_size=128,
        validation_split=0.25,
        optimizer=optimizer,
    )
    optimizer_histories[name] = history_opt
    optimizer_results.append({
        "optimizador": name,
        "val_accuracy": history_opt.history["val_accuracy"][-1],
        "val_loss": history_opt.history["val_loss"][-1],
        "test_accuracy": test_acc_opt,
        "tiempo_s": elapsed_opt,
    })

optimizer_results_df = pd.DataFrame(optimizer_results)
display(optimizer_results_df)

plt.figure(figsize=(8, 5))
for name, history in optimizer_histories.items():
    plt.plot(history.history["val_accuracy"], label=name)
plt.xlabel("epoch")
plt.ylabel("validation accuracy")
plt.title("Comparación de optimizadores")
plt.legend()
plt.show()

```

	optimizador	val_accuracy	val_loss	test_accuracy	tiempo_s
0	SGD lr=0.01	0.932800	0.228449	0.9358	8.155466
1	SGD momentum=0.9	0.969000	0.106633	0.9739	10.077158
2	RMSprop	0.976733	0.125344	0.9790	10.913715
3	Adam	0.968600	0.128665	0.9696	11.045771



Análisis de la pregunta 11.1:

SGD básico aprende, pero suele necesitar más epochs y un ajuste más cuidadoso de la tasa de aprendizaje para competir con métodos adaptativos. Al añadir momentum, SGD aprovecha parte de la dirección acumulada del gradiente y converge con más estabilidad. RMSprop y Adam adaptan el paso por parámetro, lo que en este problema permite alcanzar buenas métricas en menos tiempo. En los resultados, Adam es la opción más equilibrada entre rapidez de convergencia, `val_accuracy` y facilidad de configuración, por lo que se utilizará en la red final.

12. Regularización y red final (1.25 puntos)

****Problema 12.1 *(2 puntos)**:** Entrenar una red final que sea capaz de obtener una accuracy en el validation superior al 95%. Para ello, combinar todo lo aprendido anteriormente y utilizar técnicas de regularización para evitar overfitting. Algunos de los elementos que pueden tenerse en cuenta son los siguientes.

- Número de capas y neuronas por capa
- Optimizadores y sus parámetros
- Batch size
- Unidades de activación
- Uso de capas dropout, regularización L2, regularización L1...
- Early stopping (se puede aplicar como un callback de Keras, o se puede ver un poco "a ojo" cuándo el modelo empieza a caer en overfitting y seleccionar el número de epochs necesarias)
- Batch normalization

Si los modelos entrenados anteriormente ya se acercaban al valor requerido de accuracy, probar distintas estrategias igualmente y comentar los resultados.

Explicar brevemente la estrategia seguida y los modelos probados para obtener el modelo final, que debe verse entrenado en este Notebook. No es necesario guardar el entrenamiento de todos los modelos que se han probado, es suficiente con explicar cómo se ha llegado al modelo final.

```
In [26]: reset_seeds()
tf.keras.backend.clear_session()

final_model = create_mlp(
    hidden_layers=[512, 256, 128],
    activation="relu",
    initializer="glorot_uniform",
    dropout_rate=0.20,
    l2_value=1e-4,
    batch_norm=True,
)

final_model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=1e-3),
    loss="categorical_crossentropy",
    metrics=["accuracy"],
)

final_callbacks = [
    keras.callbacks.EarlyStopping(
        monitor="val_accuracy",
        patience=5,
        min_delta=1e-4,
        mode="max",
        restore_best_weights=True,
        verbose=1,
    ),
    keras.callbacks.ReduceLROnPlateau(
        monitor="val_loss",
        factor=0.5,
        patience=2,
        min_lr=1e-5,
        verbose=1,
    ),
]

start = time.time()
history_final = final_model.fit(
    X_train,
    y_train,
    epochs=35,
    batch_size=128,
    validation_split=0.10,
    callbacks=final_callbacks,
    verbose=1,
)

final_time = time.time() - start

plot_history(history_final, "Red final regularizada")
final_test_loss, final_test_acc = final_model.evaluate(X_test, y_test,
```



```

verbose=0)
best_val_acc = max(history_final.history["val_accuracy"])
best_val_loss = min(history_final.history["val_loss"])

print(f"Mejor validation accuracy: {best_val_acc:.4f}")
print(f"Mejor validation loss: {best_val_loss:.4f}")
print(f"Test accuracy final: {final_test_acc:.4f}")
print(f"Test loss final: {final_test_loss:.4f}")
print(f"Tiempo de entrenamiento: {final_time:.1f} segundos")

pred_probs = final_model.predict(X_test[:12], verbose=0)
pred_classes = np.argmax(pred_probs, axis=1)

fig, axes = plt.subplots(2, 6, figsize=(12, 5))
for i, ax in enumerate(axes.ravel()):
    ax.imshow(test_images_raw[i], cmap="gray")
    ax.set_title(f"Real: {test_labels_raw[i]} | Pred: {pred_classes[i]}")
    ax.axis("off")
plt.tight_layout()
plt.show()

y_pred = np.argmax(final_model.predict(X_test, verbose=0), axis=1)
cm = confusion_matrix(test_labels_raw, y_pred)
display(pd.DataFrame(cm, index=[f"real_{i}" for i in range(10)], columns=[f"pred_{i}" for i in range(10)]))
print(classification_report(test_labels_raw, y_pred, digits=4))

```

Epoch 1/35

1/422		8:55	1s/step	- accuracy: 0.1016	- loss: 3.0436
11/422		2s	5ms/step	- accuracy: 0.3274	- loss: 2.2177
20/422		2s	6ms/step	- accuracy: 0.4408	- loss: 1.8724
31/422		2s	5ms/step	- accuracy: 0.5214	- loss: 1.6290
44/422		1s	5ms/step	- accuracy: 0.5834	- loss: 1.4423
57/422		1s	5ms/step	- accuracy: 0.6265	- loss: 1.3120
69/422		1s	5ms/step	- accuracy: 0.6558	- loss: 1.2219
82/422		1s	5ms/step	- accuracy: 0.6808	- loss: 1.1445
95/422		1s	5ms/step	- accuracy: 0.7008	- loss: 1.0819
107/422		1s	5ms/step	- accuracy: 0.7162	- loss: 1.0337
118/422		1s	5ms/step	- accuracy: 0.7281	- loss: 0.9957
129/422		1s	5ms/step	- accuracy: 0.7386	- loss: 0.9622
140/422		1s	5ms/step	- accuracy: 0.7479	- loss: 0.9325
151/422		1s	5ms/step	- accuracy: 0.7562	- loss: 0.9059
162/422		1s	5ms/step	- accuracy: 0.7637	- loss: 0.8819
173/422		1s	5ms/step	- accuracy: 0.7704	- loss: 0.8600
184/422		1s	5ms/step	- accuracy: 0.7766	- loss: 0.8401

195/422  1s 5ms/step - accuracy: 0.7822 - loss: 0.8218

206/422  0s 5ms/step - accuracy: 0.7874 - loss: 0.8048

217/422  0s 5ms/step - accuracy: 0.7922 - loss: 0.7891

228/422  0s 5ms/step - accuracy: 0.7967 - loss: 0.7745

238/422  0s 5ms/step - accuracy: 0.8005 - loss: 0.7620

247/422  0s 5ms/step - accuracy: 0.8037 - loss: 0.7514

258/422  0s 5ms/step - accuracy: 0.8075 - loss: 0.7392

269/422  0s 5ms/step - accuracy: 0.8110 - loss: 0.7276

280/422  0s 5ms/step - accuracy: 0.8143 - loss: 0.7167

291/422  0s 5ms/step - accuracy: 0.8175 - loss: 0.7063

301/422  0s 5ms/step - accuracy: 0.8202 - loss: 0.6974

312/422  0s 5ms/step - accuracy: 0.8230 - loss: 0.6880

323/422  0s 5ms/step - accuracy: 0.8257 - loss: 0.6790

334/422  0s 5ms/step - accuracy: 0.8283 - loss: 0.6705

344/422  0s 5ms/step - accuracy: 0.8306 - loss: 0.6631

355/422  0s 5ms/step - accuracy: 0.8329 - loss: 0.6553

365/422  0s 5ms/step - accuracy: 0.8350 - loss: 0.6485

376/422  0s 5ms/step - accuracy: 0.8371 - loss: 0.6413

387/422  0s 5ms/step - accuracy: 0.8392 - loss: 0.6344

398/422  0s 5ms/step - accuracy: 0.8412 - loss: 0.6279

409/422  0s 5ms/step - accuracy: 0.8431 - loss: 0.6216

420/422  0s 5ms/step - accuracy: 0.8449 - loss: 0.6155

422/422  4s 5ms/step - accuracy: 0.9136 - loss: 0.3872 -
val_accuracy: 0.9712 - val_loss: 0.1935 - learning_rate: 0.0010

Epoch 2/35

1/422  6s 15ms/step - accuracy: 0.9453 - loss: 0.2633

11/422  2s 5ms/step - accuracy: 0.9495 - loss: 0.2496

21/422  2s 5ms/step - accuracy: 0.9515 - loss: 0.2496

31/422  1s 5ms/step - accuracy: 0.9511 - loss: 0.2520

42/422  1s 5ms/step - accuracy: 0.9510 - loss: 0.2533

52/422  1s 5ms/step - accuracy: 0.9513 - loss: 0.2533

63/422		1s	5ms/step	- accuracy: 0.9517	- loss: 0.2529
73/422		1s	5ms/step	- accuracy: 0.9520	- loss: 0.2525
84/422		1s	5ms/step	- accuracy: 0.9523	- loss: 0.2519
96/422		1s	5ms/step	- accuracy: 0.9525	- loss: 0.2512
107/422		1s	5ms/step	- accuracy: 0.9527	- loss: 0.2506
118/422		1s	5ms/step	- accuracy: 0.9529	- loss: 0.2501
129/422		1s	5ms/step	- accuracy: 0.9531	- loss: 0.2496
140/422		1s	5ms/step	- accuracy: 0.9532	- loss: 0.2490
151/422		1s	5ms/step	- accuracy: 0.9534	- loss: 0.2485
162/422		1s	5ms/step	- accuracy: 0.9535	- loss: 0.2479
173/422		1s	5ms/step	- accuracy: 0.9537	- loss: 0.2475
184/422		1s	5ms/step	- accuracy: 0.9538	- loss: 0.2471
195/422		1s	5ms/step	- accuracy: 0.9539	- loss: 0.2467
206/422		1s	5ms/step	- accuracy: 0.9541	- loss: 0.2462
217/422		0s	5ms/step	- accuracy: 0.9542	- loss: 0.2458
228/422		0s	5ms/step	- accuracy: 0.9543	- loss: 0.2453
239/422		0s	5ms/step	- accuracy: 0.9544	- loss: 0.2449
250/422		0s	5ms/step	- accuracy: 0.9546	- loss: 0.2444
261/422		0s	5ms/step	- accuracy: 0.9547	- loss: 0.2439
272/422		0s	5ms/step	- accuracy: 0.9548	- loss: 0.2434
283/422		0s	5ms/step	- accuracy: 0.9550	- loss: 0.2429
294/422		0s	5ms/step	- accuracy: 0.9551	- loss: 0.2424
305/422		0s	5ms/step	- accuracy: 0.9552	- loss: 0.2418
316/422		0s	5ms/step	- accuracy: 0.9554	- loss: 0.2413
327/422		0s	5ms/step	- accuracy: 0.9555	- loss: 0.2408
338/422		0s	5ms/step	- accuracy: 0.9556	- loss: 0.2403
349/422		0s	5ms/step	- accuracy: 0.9557	- loss: 0.2398
360/422		0s	5ms/step	- accuracy: 0.9559	- loss: 0.2394
371/422		0s	5ms/step	- accuracy: 0.9560	- loss: 0.2389
382/422		0s	5ms/step	- accuracy: 0.9561	- loss: 0.2385

393/422  0s 5ms/step - accuracy: 0.9562 - loss: 0.2381

404/422  0s 5ms/step - accuracy: 0.9563 - loss: 0.2377

415/422  0s 5ms/step - accuracy: 0.9564 - loss: 0.2373

422/422  2s 5ms/step - accuracy: 0.9601 - loss: 0.2224 -
val_accuracy: 0.9777 - val_loss: 0.1641 - learning_rate: 0.0010

Epoch 3/35

1/422  4s 12ms/step - accuracy: 0.9688 - loss: 0.1590

12/422  1s 5ms/step - accuracy: 0.9680 - loss: 0.1749

23/422  1s 5ms/step - accuracy: 0.9688 - loss: 0.1821

34/422  1s 5ms/step - accuracy: 0.9689 - loss: 0.1862

45/422  1s 5ms/step - accuracy: 0.9690 - loss: 0.1882

56/422  1s 5ms/step - accuracy: 0.9690 - loss: 0.1896

67/422  1s 5ms/step - accuracy: 0.9689 - loss: 0.1910

78/422  1s 5ms/step - accuracy: 0.9688 - loss: 0.1918

89/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1924

100/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1928

111/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1932

122/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1934

133/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1936

144/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1937

155/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1938

165/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1938

176/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1938

186/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1938

197/422  1s 5ms/step - accuracy: 0.9687 - loss: 0.1937

208/422  1s 5ms/step - accuracy: 0.9688 - loss: 0.1936

219/422  0s 5ms/step - accuracy: 0.9688 - loss: 0.1934

230/422  0s 5ms/step - accuracy: 0.9688 - loss: 0.1933

241/422  0s 5ms/step - accuracy: 0.9688 - loss: 0.1933

252/422  0s 5ms/step - accuracy: 0.9688 - loss: 0.1932

263/422  0s 5ms/step - accuracy: 0.9689 - loss: 0.1930

273/422  0s 5ms/step - accuracy: 0.9689 - loss: 0.1929

283/422  0s 5ms/step - accuracy: 0.9689 - loss: 0.1928

294/422  0s 5ms/step - accuracy: 0.9690 - loss: 0.1926

304/422  0s 5ms/step - accuracy: 0.9690 - loss: 0.1924

314/422  0s 5ms/step - accuracy: 0.9691 - loss: 0.1922

325/422  0s 5ms/step - accuracy: 0.9691 - loss: 0.1921

336/422  0s 5ms/step - accuracy: 0.9692 - loss: 0.1919

347/422  0s 5ms/step - accuracy: 0.9692 - loss: 0.1918

357/422  0s 5ms/step - accuracy: 0.9692 - loss: 0.1916

367/422  0s 5ms/step - accuracy: 0.9692 - loss: 0.1915

377/422  0s 5ms/step - accuracy: 0.9693 - loss: 0.1914

387/422  0s 5ms/step - accuracy: 0.9693 - loss: 0.1913

397/422  0s 5ms/step - accuracy: 0.9693 - loss: 0.1912

407/422  0s 5ms/step - accuracy: 0.9693 - loss: 0.1911

418/422  0s 5ms/step - accuracy: 0.9694 - loss: 0.1909

422/422  2s 5ms/step - accuracy: 0.9705 - loss: 0.1858 -
val_accuracy: 0.9793 - val_loss: 0.1590 - learning_rate: 0.0010
Epoch 4/35

1/422  7s 19ms/step - accuracy: 0.9609 - loss: 0.1468

12/422  1s 5ms/step - accuracy: 0.9709 - loss: 0.1668

23/422  1s 5ms/step - accuracy: 0.9716 - loss: 0.1693

34/422  1s 5ms/step - accuracy: 0.9716 - loss: 0.1712

45/422  1s 5ms/step - accuracy: 0.9719 - loss: 0.1716

56/422  1s 5ms/step - accuracy: 0.9723 - loss: 0.1714

67/422  1s 5ms/step - accuracy: 0.9726 - loss: 0.1716

78/422  1s 5ms/step - accuracy: 0.9727 - loss: 0.1719

89/422  1s 5ms/step - accuracy: 0.9728 - loss: 0.1723

100/422  1s 5ms/step - accuracy: 0.9728 - loss: 0.1728

111/422  1s 5ms/step - accuracy: 0.9728 - loss: 0.1731

122/422  1s 5ms/step - accuracy: 0.9728 - loss: 0.1734

133/422  1s 5ms/step - accuracy: 0.9728 - loss: 0.1737

144/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1738

155/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1739

166/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1739

177/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1740

188/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1740

199/422  1s 5ms/step - accuracy: 0.9729 - loss: 0.1740

211/422  0s 5ms/step - accuracy: 0.9729 - loss: 0.1740

222/422  0s 5ms/step - accuracy: 0.9729 - loss: 0.1739

233/422  0s 5ms/step - accuracy: 0.9730 - loss: 0.1738

244/422  0s 5ms/step - accuracy: 0.9730 - loss: 0.1737

255/422  0s 5ms/step - accuracy: 0.9731 - loss: 0.1736

266/422  0s 5ms/step - accuracy: 0.9731 - loss: 0.1735

277/422  0s 5ms/step - accuracy: 0.9732 - loss: 0.1733

288/422  0s 5ms/step - accuracy: 0.9732 - loss: 0.1732

299/422  0s 5ms/step - accuracy: 0.9733 - loss: 0.1730

310/422  0s 5ms/step - accuracy: 0.9734 - loss: 0.1728

321/422  0s 5ms/step - accuracy: 0.9734 - loss: 0.1726

332/422  0s 5ms/step - accuracy: 0.9735 - loss: 0.1724

343/422  0s 5ms/step - accuracy: 0.9735 - loss: 0.1722

354/422  0s 5ms/step - accuracy: 0.9736 - loss: 0.1720

365/422  0s 5ms/step - accuracy: 0.9736 - loss: 0.1718

376/422  0s 5ms/step - accuracy: 0.9737 - loss: 0.1717

387/422  0s 5ms/step - accuracy: 0.9737 - loss: 0.1716

398/422  0s 5ms/step - accuracy: 0.9737 - loss: 0.1714

404/422  0s 5ms/step - accuracy: 0.9738 - loss: 0.1714

413/422  0s 5ms/step - accuracy: 0.9738 - loss: 0.1713

422/422  2s 5ms/step - accuracy: 0.9752 - loss: 0.1665 -
val_accuracy: 0.9768 - val_loss: 0.1631 - learning_rate: 0.0010
Epoch 5/35

1/422  5s 13ms/step - accuracy: 0.9922 - loss: 0.1206

12/422  1s 5ms/step - accuracy: 0.9857 - loss: 0.1346

23/422		1s	5ms/step	- accuracy: 0.9839	- loss: 0.1392
34/422		1s	5ms/step	- accuracy: 0.9816	- loss: 0.1447
45/422		1s	5ms/step	- accuracy: 0.9806	- loss: 0.1480
56/422		1s	5ms/step	- accuracy: 0.9800	- loss: 0.1503
67/422		1s	5ms/step	- accuracy: 0.9795	- loss: 0.1523
78/422		1s	5ms/step	- accuracy: 0.9792	- loss: 0.1537
89/422		1s	5ms/step	- accuracy: 0.9789	- loss: 0.1547
99/422		1s	5ms/step	- accuracy: 0.9787	- loss: 0.1555
109/422		1s	5ms/step	- accuracy: 0.9784	- loss: 0.1563
119/422		1s	5ms/step	- accuracy: 0.9783	- loss: 0.1568
129/422		1s	5ms/step	- accuracy: 0.9781	- loss: 0.1572
139/422		1s	5ms/step	- accuracy: 0.9780	- loss: 0.1575
150/422		1s	5ms/step	- accuracy: 0.9779	- loss: 0.1578
160/422		1s	5ms/step	- accuracy: 0.9778	- loss: 0.1580
170/422		1s	5ms/step	- accuracy: 0.9778	- loss: 0.1582
180/422		1s	5ms/step	- accuracy: 0.9777	- loss: 0.1583
190/422		1s	5ms/step	- accuracy: 0.9776	- loss: 0.1584
200/422		1s	5ms/step	- accuracy: 0.9776	- loss: 0.1585
210/422		1s	5ms/step	- accuracy: 0.9776	- loss: 0.1585
221/422		0s	5ms/step	- accuracy: 0.9776	- loss: 0.1586
231/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
241/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
251/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
261/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
271/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
281/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1586
290/422		0s	5ms/step	- accuracy: 0.9775	- loss: 0.1585
300/422		0s	5ms/step	- accuracy: 0.9776	- loss: 0.1584
310/422		0s	5ms/step	- accuracy: 0.9776	- loss: 0.1583
320/422		0s	5ms/step	- accuracy: 0.9776	- loss: 0.1582

330/422  0s 5ms/step - accuracy: 0.9776 - loss: 0.1581

340/422  0s 5ms/step - accuracy: 0.9776 - loss: 0.1580

350/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1579

360/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1578

371/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1577

382/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1577

393/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1576

403/422  0s 5ms/step - accuracy: 0.9777 - loss: 0.1575

414/422  0s 5ms/step - accuracy: 0.9778 - loss: 0.1575

422/422  2s 5ms/step - accuracy: 0.9782 - loss: 0.1556 -
val_accuracy: 0.9792 - val_loss: 0.1550 - learning_rate: 0.0010

Epoch 6/35

1/422  7s 18ms/step - accuracy: 0.9766 - loss: 0.1452

9/422  2s 6ms/step - accuracy: 0.9803 - loss: 0.1414

17/422  2s 7ms/step - accuracy: 0.9802 - loss: 0.1442

24/422  2s 7ms/step - accuracy: 0.9797 - loss: 0.1462

31/422  2s 7ms/step - accuracy: 0.9792 - loss: 0.1478

39/422  2s 7ms/step - accuracy: 0.9790 - loss: 0.1490

47/422  2s 7ms/step - accuracy: 0.9790 - loss: 0.1494

54/422  2s 7ms/step - accuracy: 0.9790 - loss: 0.1497

62/422  2s 7ms/step - accuracy: 0.9789 - loss: 0.1499

71/422  2s 7ms/step - accuracy: 0.9789 - loss: 0.1502

80/422  2s 7ms/step - accuracy: 0.9789 - loss: 0.1504

88/422  2s 7ms/step - accuracy: 0.9788 - loss: 0.1505

96/422  2s 7ms/step - accuracy: 0.9787 - loss: 0.1506

105/422  2s 7ms/step - accuracy: 0.9787 - loss: 0.1507

113/422  2s 7ms/step - accuracy: 0.9786 - loss: 0.1508

121/422  2s 7ms/step - accuracy: 0.9786 - loss: 0.1508

128/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1508

134/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1508

145/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1508

156/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1508

167/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1507

178/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1506

188/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1505

194/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1505

197/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1504

201/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1504

209/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1503

220/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1502

231/422  1s 7ms/step - accuracy: 0.9786 - loss: 0.1501

242/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1500

253/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1499

263/422  1s 6ms/step - accuracy: 0.9786 - loss: 0.1498

273/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1497

284/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1497

293/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1496

304/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1495

315/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1494

326/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1494

337/422  0s 6ms/step - accuracy: 0.9786 - loss: 0.1493

347/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1492

355/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1491

366/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1490

376/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1489

386/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1488

396/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1488

407/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1487

418/422  0s 6ms/step - accuracy: 0.9787 - loss: 0.1486

422/422  3s 6ms/step - accuracy: 0.9792 - loss: 0.1464 -
val_accuracy: 0.9830 - val_loss: 0.1394 - learning_rate: 0.0010
Epoch 7/35

1/422		5s	14ms/step	- accuracy: 1.0000	- loss: 0.1111
11/422		2s	5ms/step	- accuracy: 0.9849	- loss: 0.1223
22/422		1s	5ms/step	- accuracy: 0.9819	- loss: 0.1295
33/422		1s	5ms/step	- accuracy: 0.9808	- loss: 0.1325
43/422		1s	5ms/step	- accuracy: 0.9804	- loss: 0.1338
53/422		1s	5ms/step	- accuracy: 0.9803	- loss: 0.1346
63/422		1s	5ms/step	- accuracy: 0.9803	- loss: 0.1352
74/422		1s	5ms/step	- accuracy: 0.9802	- loss: 0.1360
85/422		1s	5ms/step	- accuracy: 0.9800	- loss: 0.1367
96/422		1s	5ms/step	- accuracy: 0.9799	- loss: 0.1374
107/422		1s	5ms/step	- accuracy: 0.9798	- loss: 0.1381
118/422		1s	5ms/step	- accuracy: 0.9797	- loss: 0.1387
128/422		1s	5ms/step	- accuracy: 0.9796	- loss: 0.1391
138/422		1s	5ms/step	- accuracy: 0.9796	- loss: 0.1394
149/422		1s	5ms/step	- accuracy: 0.9795	- loss: 0.1397
160/422		1s	5ms/step	- accuracy: 0.9795	- loss: 0.1399
171/422		1s	5ms/step	- accuracy: 0.9795	- loss: 0.1401
182/422		1s	5ms/step	- accuracy: 0.9794	- loss: 0.1403
193/422		1s	5ms/step	- accuracy: 0.9794	- loss: 0.1405
203/422		1s	5ms/step	- accuracy: 0.9794	- loss: 0.1406
213/422		1s	5ms/step	- accuracy: 0.9794	- loss: 0.1407
223/422		0s	5ms/step	- accuracy: 0.9794	- loss: 0.1407
234/422		0s	5ms/step	- accuracy: 0.9795	- loss: 0.1408
245/422		0s	5ms/step	- accuracy: 0.9795	- loss: 0.1408
256/422		0s	5ms/step	- accuracy: 0.9795	- loss: 0.1408
267/422		0s	5ms/step	- accuracy: 0.9795	- loss: 0.1408
278/422		0s	5ms/step	- accuracy: 0.9796	- loss: 0.1408
289/422		0s	5ms/step	- accuracy: 0.9796	- loss: 0.1408
300/422		0s	5ms/step	- accuracy: 0.9796	- loss: 0.1408
311/422		0s	5ms/step	- accuracy: 0.9797	- loss: 0.1407

322/422  0s 5ms/step - accuracy: 0.9797 - loss: 0.1407

333/422  0s 5ms/step - accuracy: 0.9797 - loss: 0.1406

344/422  0s 5ms/step - accuracy: 0.9798 - loss: 0.1406

355/422  0s 5ms/step - accuracy: 0.9798 - loss: 0.1405

366/422  0s 5ms/step - accuracy: 0.9798 - loss: 0.1404

377/422  0s 5ms/step - accuracy: 0.9799 - loss: 0.1404

388/422  0s 5ms/step - accuracy: 0.9799 - loss: 0.1404

399/422  0s 5ms/step - accuracy: 0.9799 - loss: 0.1403

410/422  0s 5ms/step - accuracy: 0.9799 - loss: 0.1403

421/422  0s 5ms/step - accuracy: 0.9799 - loss: 0.1403

422/422  2s 5ms/step - accuracy: 0.9809 - loss: 0.1394 -
val_accuracy: 0.9815 - val_loss: 0.1495 - learning_rate: 0.0010
Epoch 8/35

1/422  4s 12ms/step - accuracy: 0.9844 - loss: 0.1167

12/422  1s 5ms/step - accuracy: 0.9844 - loss: 0.1226

23/422  1s 5ms/step - accuracy: 0.9848 - loss: 0.1234

34/422  1s 5ms/step - accuracy: 0.9846 - loss: 0.1253

45/422  1s 5ms/step - accuracy: 0.9844 - loss: 0.1267

56/422  1s 5ms/step - accuracy: 0.9842 - loss: 0.1276

67/422  1s 5ms/step - accuracy: 0.9839 - loss: 0.1285

78/422  1s 5ms/step - accuracy: 0.9837 - loss: 0.1294

89/422  1s 5ms/step - accuracy: 0.9835 - loss: 0.1301

100/422  1s 5ms/step - accuracy: 0.9834 - loss: 0.1306

111/422  1s 5ms/step - accuracy: 0.9833 - loss: 0.1310

122/422  1s 5ms/step - accuracy: 0.9832 - loss: 0.1312

133/422  1s 5ms/step - accuracy: 0.9832 - loss: 0.1314

144/422  1s 5ms/step - accuracy: 0.9831 - loss: 0.1315

155/422  1s 5ms/step - accuracy: 0.9831 - loss: 0.1316

166/422  1s 5ms/step - accuracy: 0.9831 - loss: 0.1317

177/422  1s 5ms/step - accuracy: 0.9830 - loss: 0.1318

188/422  1s 5ms/step - accuracy: 0.9829 - loss: 0.1320

199/422  1s 5ms/step - accuracy: 0.9829 - loss: 0.1321

210/422  0s 5ms/step - accuracy: 0.9829 - loss: 0.1321

221/422  0s 5ms/step - accuracy: 0.9828 - loss: 0.1322

232/422  0s 5ms/step - accuracy: 0.9828 - loss: 0.1322

243/422  0s 5ms/step - accuracy: 0.9827 - loss: 0.1322

254/422  0s 5ms/step - accuracy: 0.9827 - loss: 0.1323

265/422  0s 5ms/step - accuracy: 0.9827 - loss: 0.1323

276/422  0s 5ms/step - accuracy: 0.9827 - loss: 0.1324

287/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1324

298/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1324

309/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1325

320/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1325

330/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1325

340/422  0s 5ms/step - accuracy: 0.9826 - loss: 0.1326

351/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1326

362/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1326

373/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1327

384/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1327

395/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1327

406/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1328

417/422  0s 5ms/step - accuracy: 0.9825 - loss: 0.1328

422/422  2s 5ms/step - accuracy: 0.9821 - loss: 0.1338 -
val_accuracy: 0.9818 - val_loss: 0.1383 - learning_rate: 0.0010
Epoch 9/35

1/422  5s 13ms/step - accuracy: 0.9844 - loss: 0.1144

11/422  2s 5ms/step - accuracy: 0.9821 - loss: 0.1226

22/422  1s 5ms/step - accuracy: 0.9830 - loss: 0.1233

33/422  1s 5ms/step - accuracy: 0.9828 - loss: 0.1258

44/422  1s 5ms/step - accuracy: 0.9828 - loss: 0.1272

54/422  1s 5ms/step - accuracy: 0.9829 - loss: 0.1278

64/422  1s 5ms/step - accuracy: 0.9830 - loss: 0.1283

74/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1289
83/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1293
93/422		1s	5ms/step	-	accuracy: 0.9828	-	loss: 0.1295
103/422		1s	5ms/step	-	accuracy: 0.9828	-	loss: 0.1297
113/422		1s	5ms/step	-	accuracy: 0.9828	-	loss: 0.1298
124/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1298
135/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1298
145/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1298
156/422		1s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1299
166/422		1s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1299
177/422		1s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1299
188/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1300
199/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1300
210/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1301
221/422		1s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1301
232/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1301
243/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
254/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
265/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
276/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
286/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
297/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
308/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
319/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
330/422		0s	5ms/step	-	accuracy: 0.9829	-	loss: 0.1302
341/422		0s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1302
352/422		0s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1302
363/422		0s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1302
374/422		0s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1302
385/422		0s	5ms/step	-	accuracy: 0.9830	-	loss: 0.1302

395/422  0s 5ms/step - accuracy: 0.9830 - loss: 0.1302

406/422  0s 5ms/step - accuracy: 0.9830 - loss: 0.1302

417/422  0s 5ms/step - accuracy: 0.9830 - loss: 0.1302

422/422  2s 5ms/step - accuracy: 0.9831 - loss: 0.1306 -
val_accuracy: 0.9832 - val_loss: 0.1397 - learning_rate: 0.0010

Epoch 10/35

1/422  5s 13ms/step - accuracy: 1.0000 - loss: 0.0974

12/422  1s 5ms/step - accuracy: 0.9941 - loss: 0.1077

23/422  1s 5ms/step - accuracy: 0.9910 - loss: 0.1134

34/422  1s 5ms/step - accuracy: 0.9893 - loss: 0.1171

45/422  1s 5ms/step - accuracy: 0.9884 - loss: 0.1192

56/422  1s 5ms/step - accuracy: 0.9877 - loss: 0.1209

67/422  1s 5ms/step - accuracy: 0.9872 - loss: 0.1223

78/422  1s 5ms/step - accuracy: 0.9868 - loss: 0.1231

89/422  1s 5ms/step - accuracy: 0.9866 - loss: 0.1235

100/422  1s 5ms/step - accuracy: 0.9864 - loss: 0.1238

111/422  1s 5ms/step - accuracy: 0.9862 - loss: 0.1241

122/422  1s 5ms/step - accuracy: 0.9860 - loss: 0.1244

133/422  1s 5ms/step - accuracy: 0.9859 - loss: 0.1247

144/422  1s 5ms/step - accuracy: 0.9857 - loss: 0.1250

155/422  1s 5ms/step - accuracy: 0.9856 - loss: 0.1251

166/422  1s 5ms/step - accuracy: 0.9856 - loss: 0.1253

177/422  1s 5ms/step - accuracy: 0.9855 - loss: 0.1254

188/422  1s 5ms/step - accuracy: 0.9855 - loss: 0.1255

199/422  1s 5ms/step - accuracy: 0.9854 - loss: 0.1256

210/422  0s 5ms/step - accuracy: 0.9854 - loss: 0.1256

221/422  0s 5ms/step - accuracy: 0.9853 - loss: 0.1257

232/422  0s 5ms/step - accuracy: 0.9853 - loss: 0.1258

243/422  0s 5ms/step - accuracy: 0.9853 - loss: 0.1258

254/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1259

265/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1259

275/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1259

285/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1259

296/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1259

307/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1259

317/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1258

327/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1258

337/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1258

347/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1257

357/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1257

368/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1257

378/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1256

388/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1256

398/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1256

408/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1256

418/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1255

422/422  2s 5ms/step - accuracy: 0.9850 - loss: 0.1246 -
val_accuracy: 0.9870 - val_loss: 0.1324 - learning_rate: 0.0010
Epoch 11/35

1/422  5s 14ms/step - accuracy: 0.9922 - loss: 0.1095

11/422  2s 5ms/step - accuracy: 0.9923 - loss: 0.1060

21/422  2s 5ms/step - accuracy: 0.9914 - loss: 0.1082

31/422  2s 5ms/step - accuracy: 0.9908 - loss: 0.1100

41/422  2s 5ms/step - accuracy: 0.9905 - loss: 0.1110

51/422  1s 5ms/step - accuracy: 0.9904 - loss: 0.1115

61/422  1s 5ms/step - accuracy: 0.9901 - loss: 0.1121

71/422  1s 5ms/step - accuracy: 0.9898 - loss: 0.1129

81/422  1s 5ms/step - accuracy: 0.9894 - loss: 0.1138

91/422  1s 5ms/step - accuracy: 0.9890 - loss: 0.1147

100/422  1s 5ms/step - accuracy: 0.9887 - loss: 0.1156

107/422  1s 6ms/step - accuracy: 0.9885 - loss: 0.1162

114/422  1s 6ms/step - accuracy: 0.9882 - loss: 0.1168

118/422		1s 6ms/step	- accuracy: 0.9881	- loss: 0.1171
123/422		1s 6ms/step	- accuracy: 0.9879	- loss: 0.1175
132/422		1s 6ms/step	- accuracy: 0.9877	- loss: 0.1182
139/422		1s 6ms/step	- accuracy: 0.9875	- loss: 0.1187
148/422		1s 6ms/step	- accuracy: 0.9873	- loss: 0.1192
157/422		1s 6ms/step	- accuracy: 0.9871	- loss: 0.1197
164/422		1s 6ms/step	- accuracy: 0.9870	- loss: 0.1199
168/422		1s 6ms/step	- accuracy: 0.9870	- loss: 0.1201
176/422		1s 6ms/step	- accuracy: 0.9868	- loss: 0.1204
184/422		1s 6ms/step	- accuracy: 0.9867	- loss: 0.1207
193/422		1s 6ms/step	- accuracy: 0.9866	- loss: 0.1210
202/422		1s 6ms/step	- accuracy: 0.9865	- loss: 0.1212
211/422		1s 6ms/step	- accuracy: 0.9864	- loss: 0.1214
218/422		1s 6ms/step	- accuracy: 0.9863	- loss: 0.1216
228/422		1s 6ms/step	- accuracy: 0.9862	- loss: 0.1218
239/422		1s 6ms/step	- accuracy: 0.9861	- loss: 0.1220
249/422		1s 6ms/step	- accuracy: 0.9861	- loss: 0.1222
260/422		1s 6ms/step	- accuracy: 0.9860	- loss: 0.1224
268/422		0s 6ms/step	- accuracy: 0.9859	- loss: 0.1225
277/422		0s 6ms/step	- accuracy: 0.9859	- loss: 0.1227
285/422		0s 6ms/step	- accuracy: 0.9858	- loss: 0.1228
296/422		0s 6ms/step	- accuracy: 0.9858	- loss: 0.1229
307/422		0s 6ms/step	- accuracy: 0.9857	- loss: 0.1230
317/422		0s 6ms/step	- accuracy: 0.9857	- loss: 0.1231
328/422		0s 6ms/step	- accuracy: 0.9856	- loss: 0.1232
339/422		0s 6ms/step	- accuracy: 0.9856	- loss: 0.1233
350/422		0s 6ms/step	- accuracy: 0.9856	- loss: 0.1234
360/422		0s 6ms/step	- accuracy: 0.9855	- loss: 0.1234
371/422		0s 6ms/step	- accuracy: 0.9855	- loss: 0.1235
380/422		0s 6ms/step	- accuracy: 0.9855	- loss: 0.1235

390/422  0s 6ms/step - accuracy: 0.9855 - loss: 0.1236

396/422  0s 6ms/step - accuracy: 0.9854 - loss: 0.1236

403/422  0s 6ms/step - accuracy: 0.9854 - loss: 0.1236

412/422  0s 6ms/step - accuracy: 0.9854 - loss: 0.1237

420/422  0s 6ms/step - accuracy: 0.9854 - loss: 0.1237

422/422  3s 6ms/step - accuracy: 0.9846 - loss: 0.1258 -
val_accuracy: 0.9800 - val_loss: 0.1527 - learning_rate: 0.0010
Epoch 12/35

1/422  5s 14ms/step - accuracy: 0.9844 - loss: 0.1048

10/422  2s 6ms/step - accuracy: 0.9862 - loss: 0.1122

18/422  2s 6ms/step - accuracy: 0.9855 - loss: 0.1166

27/422  2s 6ms/step - accuracy: 0.9853 - loss: 0.1198

35/422  2s 6ms/step - accuracy: 0.9852 - loss: 0.1214

43/422  2s 6ms/step - accuracy: 0.9853 - loss: 0.1221

54/422  2s 6ms/step - accuracy: 0.9854 - loss: 0.1222

65/422  2s 6ms/step - accuracy: 0.9854 - loss: 0.1222

73/422  2s 6ms/step - accuracy: 0.9853 - loss: 0.1223

82/422  2s 6ms/step - accuracy: 0.9852 - loss: 0.1224

90/422  1s 6ms/step - accuracy: 0.9851 - loss: 0.1224

99/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1224

109/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1223

120/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1223

131/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1223

141/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1223

151/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1223

160/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1224

170/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1224

180/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1225

190/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1225

201/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1225

211/422  1s 6ms/step - accuracy: 0.9850 - loss: 0.1225

222/422  1s 5ms/step - accuracy: 0.9850 - loss: 0.1225

232/422  1s 5ms/step - accuracy: 0.9850 - loss: 0.1225

243/422  0s 5ms/step - accuracy: 0.9850 - loss: 0.1225

254/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1225

265/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1224

273/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1224

283/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1224

294/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1223

305/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1222

316/422  0s 5ms/step - accuracy: 0.9851 - loss: 0.1222

327/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1221

338/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1220

348/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1220

359/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1219

370/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1219

381/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1219

392/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1219

404/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1218

415/422  0s 5ms/step - accuracy: 0.9852 - loss: 0.1218

Epoch 12: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257.

422/422  2s 5ms/step - accuracy: 0.9852 - loss: 0.1217 - val_accuracy: 0.9797 - val_loss: 0.1529 - learning_rate: 0.0010

Epoch 13/35

1/422  5s 12ms/step - accuracy: 0.9844 - loss: 0.1326

12/422  1s 5ms/step - accuracy: 0.9896 - loss: 0.1139

23/422  1s 5ms/step - accuracy: 0.9891 - loss: 0.1147

34/422  1s 5ms/step - accuracy: 0.9888 - loss: 0.1140

45/422  1s 5ms/step - accuracy: 0.9887 - loss: 0.1133

56/422  1s 5ms/step - accuracy: 0.9888 - loss: 0.1129

67/422  1s 5ms/step - accuracy: 0.9888 - loss: 0.1128

78/422  1s 5ms/step - accuracy: 0.9889 - loss: 0.1125

89/422  1s 5ms/step - accuracy: 0.9889 - loss: 0.1123

100/422		1s	5ms/step	-	accuracy: 0.9890	-	loss: 0.1122
111/422		1s	5ms/step	-	accuracy: 0.9890	-	loss: 0.1121
122/422		1s	5ms/step	-	accuracy: 0.9890	-	loss: 0.1121
133/422		1s	5ms/step	-	accuracy: 0.9891	-	loss: 0.1119
144/422		1s	5ms/step	-	accuracy: 0.9891	-	loss: 0.1118
155/422		1s	5ms/step	-	accuracy: 0.9891	-	loss: 0.1117
166/422		1s	5ms/step	-	accuracy: 0.9891	-	loss: 0.1115
177/422		1s	5ms/step	-	accuracy: 0.9892	-	loss: 0.1114
188/422		1s	5ms/step	-	accuracy: 0.9892	-	loss: 0.1112
199/422		1s	5ms/step	-	accuracy: 0.9892	-	loss: 0.1111
210/422		0s	5ms/step	-	accuracy: 0.9893	-	loss: 0.1109
221/422		0s	5ms/step	-	accuracy: 0.9893	-	loss: 0.1108
232/422		0s	5ms/step	-	accuracy: 0.9893	-	loss: 0.1107
243/422		0s	5ms/step	-	accuracy: 0.9893	-	loss: 0.1106
254/422		0s	5ms/step	-	accuracy: 0.9893	-	loss: 0.1104
265/422		0s	5ms/step	-	accuracy: 0.9894	-	loss: 0.1103
276/422		0s	5ms/step	-	accuracy: 0.9894	-	loss: 0.1101
287/422		0s	5ms/step	-	accuracy: 0.9895	-	loss: 0.1099
298/422		0s	5ms/step	-	accuracy: 0.9895	-	loss: 0.1097
309/422		0s	5ms/step	-	accuracy: 0.9896	-	loss: 0.1096
320/422		0s	5ms/step	-	accuracy: 0.9896	-	loss: 0.1094
331/422		0s	5ms/step	-	accuracy: 0.9897	-	loss: 0.1092
342/422		0s	5ms/step	-	accuracy: 0.9897	-	loss: 0.1090
353/422		0s	5ms/step	-	accuracy: 0.9898	-	loss: 0.1088
364/422		0s	5ms/step	-	accuracy: 0.9898	-	loss: 0.1086
374/422		0s	5ms/step	-	accuracy: 0.9899	-	loss: 0.1084
385/422		0s	5ms/step	-	accuracy: 0.9899	-	loss: 0.1082
396/422		0s	5ms/step	-	accuracy: 0.9900	-	loss: 0.1080
407/422		0s	5ms/step	-	accuracy: 0.9900	-	loss: 0.1078
418/422		0s	5ms/step	-	accuracy: 0.9901	-	loss: 0.1077

422/422  2s 5ms/step - accuracy: 0.9919 - loss: 0.1009 - val_accuracy: 0.9875 - val_loss: 0.1246 - learning_rate: 5.0000e-04
Epoch 14/35

1/422		5s 13ms/step - accuracy: 0.9922 - loss: 0.0820
12/422		1s 5ms/step - accuracy: 0.9897 - loss: 0.1011
23/422	 	1s 5ms/step - accuracy: 0.9901 - loss: 0.0986
33/422	 	1s 5ms/step - accuracy: 0.9903 - loss: 0.0979
44/422	 	1s 5ms/step - accuracy: 0.9904 - loss: 0.0975
54/422	 	1s 5ms/step - accuracy: 0.9905 - loss: 0.0971
64/422	 	1s 5ms/step - accuracy: 0.9907 - loss: 0.0968
75/422	 	1s 5ms/step - accuracy: 0.9908 - loss: 0.0966
86/422	 	1s 5ms/step - accuracy: 0.9909 - loss: 0.0964
97/422	 	1s 5ms/step - accuracy: 0.9910 - loss: 0.0963
108/422	 	1s 5ms/step - accuracy: 0.9911 - loss: 0.0962
119/422	 	1s 5ms/step - accuracy: 0.9912 - loss: 0.0961
129/422	 	1s 5ms/step - accuracy: 0.9913 - loss: 0.0959
140/422	 	1s 5ms/step - accuracy: 0.9914 - loss: 0.0958
150/422	 	1s 5ms/step - accuracy: 0.9915 - loss: 0.0956
161/422	 	1s 5ms/step - accuracy: 0.9915 - loss: 0.0955
172/422	 	1s 5ms/step - accuracy: 0.9916 - loss: 0.0953
183/422	 	1s 5ms/step - accuracy: 0.9917 - loss: 0.0951
193/422	 	1s 5ms/step - accuracy: 0.9917 - loss: 0.0950
204/422	 	1s 5ms/step - accuracy: 0.9918 - loss: 0.0948
214/422	 	1s 5ms/step - accuracy: 0.9918 - loss: 0.0947
224/422	 	0s 5ms/step - accuracy: 0.9919 - loss: 0.0946
235/422	 	0s 5ms/step - accuracy: 0.9919 - loss: 0.0944
246/422	 	0s 5ms/step - accuracy: 0.9920 - loss: 0.0943
257/422	 	0s 5ms/step - accuracy: 0.9920 - loss: 0.0942
268/422	 	0s 5ms/step - accuracy: 0.9921 - loss: 0.0940
279/422	 	0s 5ms/step - accuracy: 0.9921 - loss: 0.0939
290/422	 	0s 5ms/step - accuracy: 0.9921 - loss: 0.0938

301/422  0s 5ms/step - accuracy: 0.9922 - loss: 0.0937

312/422  0s 5ms/step - accuracy: 0.9922 - loss: 0.0935

323/422  0s 5ms/step - accuracy: 0.9922 - loss: 0.0934

334/422  0s 5ms/step - accuracy: 0.9923 - loss: 0.0933

345/422  0s 5ms/step - accuracy: 0.9923 - loss: 0.0932

357/422  0s 5ms/step - accuracy: 0.9923 - loss: 0.0931

369/422  0s 5ms/step - accuracy: 0.9924 - loss: 0.0930

380/422  0s 5ms/step - accuracy: 0.9924 - loss: 0.0929

391/422  0s 5ms/step - accuracy: 0.9924 - loss: 0.0928

402/422  0s 5ms/step - accuracy: 0.9925 - loss: 0.0927

413/422  0s 5ms/step - accuracy: 0.9925 - loss: 0.0926

422/422  2s 5ms/step - accuracy: 0.9934 - loss: 0.0890 -
val_accuracy: 0.9875 - val_loss: 0.1221 - learning_rate: 5.0000e-04
Epoch 15/35

1/422  5s 12ms/step - accuracy: 0.9922 - loss: 0.0864

12/422  1s 5ms/step - accuracy: 0.9956 - loss: 0.0797

23/422  1s 5ms/step - accuracy: 0.9957 - loss: 0.0800

34/422  1s 5ms/step - accuracy: 0.9957 - loss: 0.0809

45/422  1s 5ms/step - accuracy: 0.9955 - loss: 0.0814

56/422  1s 5ms/step - accuracy: 0.9953 - loss: 0.0818

67/422  1s 5ms/step - accuracy: 0.9951 - loss: 0.0825

78/422  1s 5ms/step - accuracy: 0.9948 - loss: 0.0834

89/422  1s 5ms/step - accuracy: 0.9946 - loss: 0.0840

100/422  1s 5ms/step - accuracy: 0.9945 - loss: 0.0845

111/422  1s 5ms/step - accuracy: 0.9943 - loss: 0.0849

122/422  1s 5ms/step - accuracy: 0.9942 - loss: 0.0851

133/422  1s 5ms/step - accuracy: 0.9941 - loss: 0.0853

144/422  1s 5ms/step - accuracy: 0.9941 - loss: 0.0855

155/422  1s 5ms/step - accuracy: 0.9940 - loss: 0.0856

166/422  1s 5ms/step - accuracy: 0.9940 - loss: 0.0857

177/422  1s 5ms/step - accuracy: 0.9940 - loss: 0.0858

188/422  1s 5ms/step - accuracy: 0.9939 - loss: 0.0859

199/422  1s 5ms/step - accuracy: 0.9939 - loss: 0.0860

210/422  0s 5ms/step - accuracy: 0.9939 - loss: 0.0860

221/422  0s 5ms/step - accuracy: 0.9939 - loss: 0.0860

232/422  0s 5ms/step - accuracy: 0.9939 - loss: 0.0860

243/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

254/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

265/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

276/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

287/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

298/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0861

309/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0860

320/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0860

332/422  0s 5ms/step - accuracy: 0.9938 - loss: 0.0860

344/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0860

356/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0860

367/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0860

378/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0859

389/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0859

400/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0859

411/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0859

422/422  0s 5ms/step - accuracy: 0.9937 - loss: 0.0859

422/422  2s 5ms/step - accuracy: 0.9933 - loss: 0.0853 -
val_accuracy: 0.9855 - val_loss: 0.1248 - learning_rate: 5.0000e-04

Epoch 16/35

1/422  5s 12ms/step - accuracy: 1.0000 - loss: 0.0668

12/422  2s 5ms/step - accuracy: 0.9970 - loss: 0.0734

22/422  2s 5ms/step - accuracy: 0.9957 - loss: 0.0762

32/422  1s 5ms/step - accuracy: 0.9950 - loss: 0.0774

42/422  1s 5ms/step - accuracy: 0.9947 - loss: 0.0781

52/422  1s 5ms/step - accuracy: 0.9944 - loss: 0.0788

62/422		1s	5ms/step	-	accuracy: 0.9940	-	loss: 0.0795
73/422		1s	5ms/step	-	accuracy: 0.9937	-	loss: 0.0802
84/422		1s	5ms/step	-	accuracy: 0.9935	-	loss: 0.0807
94/422		1s	5ms/step	-	accuracy: 0.9933	-	loss: 0.0811
104/422		1s	5ms/step	-	accuracy: 0.9932	-	loss: 0.0815
114/422		1s	5ms/step	-	accuracy: 0.9931	-	loss: 0.0818
124/422		1s	5ms/step	-	accuracy: 0.9930	-	loss: 0.0820
135/422		1s	5ms/step	-	accuracy: 0.9930	-	loss: 0.0823
145/422		1s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0825
155/422		1s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0827
165/422		1s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0828
176/422		1s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0829
185/422		1s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0830
195/422		1s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0831
205/422		1s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0832
215/422		1s	5ms/step	-	accuracy: 0.9927	-	loss: 0.0832
225/422		0s	5ms/step	-	accuracy: 0.9927	-	loss: 0.0832
235/422		0s	5ms/step	-	accuracy: 0.9927	-	loss: 0.0832
245/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0832
255/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0832
265/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0832
276/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0832
287/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0831
298/422		0s	5ms/step	-	accuracy: 0.9928	-	loss: 0.0831
309/422		0s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0831
320/422		0s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0830
331/422		0s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0830
342/422		0s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0829
353/422		0s	5ms/step	-	accuracy: 0.9929	-	loss: 0.0829
364/422		0s	5ms/step	-	accuracy: 0.9930	-	loss: 0.0828

375/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0828

386/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0827

397/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0827

408/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0826

419/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0826

422/422  2s 5ms/step - accuracy: 0.9934 - loss: 0.0814 -
val_accuracy: 0.9857 - val_loss: 0.1171 - learning_rate: 5.0000e-04
Epoch 17/35

1/422  5s 12ms/step - accuracy: 0.9922 - loss: 0.0769

12/422  2s 5ms/step - accuracy: 0.9915 - loss: 0.0786

23/422  1s 5ms/step - accuracy: 0.9918 - loss: 0.0788

34/422  1s 5ms/step - accuracy: 0.9918 - loss: 0.0790

45/422  1s 5ms/step - accuracy: 0.9921 - loss: 0.0787

56/422  1s 5ms/step - accuracy: 0.9924 - loss: 0.0784

67/422  1s 5ms/step - accuracy: 0.9925 - loss: 0.0784

78/422  1s 5ms/step - accuracy: 0.9926 - loss: 0.0784

89/422  1s 5ms/step - accuracy: 0.9926 - loss: 0.0785

100/422  1s 5ms/step - accuracy: 0.9926 - loss: 0.0786

111/422  1s 5ms/step - accuracy: 0.9927 - loss: 0.0787

122/422  1s 5ms/step - accuracy: 0.9927 - loss: 0.0787

133/422  1s 5ms/step - accuracy: 0.9928 - loss: 0.0787

144/422  1s 5ms/step - accuracy: 0.9928 - loss: 0.0787

154/422  1s 5ms/step - accuracy: 0.9928 - loss: 0.0787

165/422  1s 5ms/step - accuracy: 0.9929 - loss: 0.0788

176/422  1s 5ms/step - accuracy: 0.9929 - loss: 0.0787

187/422  1s 5ms/step - accuracy: 0.9929 - loss: 0.0787

197/422  1s 5ms/step - accuracy: 0.9930 - loss: 0.0787

208/422  1s 5ms/step - accuracy: 0.9930 - loss: 0.0787

219/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0787

230/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0787

241/422  0s 5ms/step - accuracy: 0.9930 - loss: 0.0787

252/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

263/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

273/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

284/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

295/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

306/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0786

317/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0785

327/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0785

336/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0785

340/422  0s 5ms/step - accuracy: 0.9931 - loss: 0.0785

343/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

345/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

353/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

361/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

369/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

377/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

385/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

392/422  0s 5ms/step - accuracy: 0.9932 - loss: 0.0785

400/422  0s 6ms/step - accuracy: 0.9932 - loss: 0.0785

408/422  0s 6ms/step - accuracy: 0.9932 - loss: 0.0785

416/422  0s 6ms/step - accuracy: 0.9932 - loss: 0.0785

422/422  2s 6ms/step - accuracy: 0.9932 - loss: 0.0786 -
val_accuracy: 0.9857 - val_loss: 0.1225 - learning_rate: 5.0000e-04
Epoch 18/35

1/422  9s 22ms/step - accuracy: 1.0000 - loss: 0.0694

7/422  4s 11ms/step - accuracy: 0.9953 - loss: 0.0718

8/422  7s 19ms/step - accuracy: 0.9953 - loss: 0.0718

16/422  5s 12ms/step - accuracy: 0.9953 - loss: 0.0718

25/422  3s 10ms/step - accuracy: 0.9949 - loss: 0.0725

34/422  3s 9ms/step - accuracy: 0.9947 - loss: 0.0728

39/422  3s 9ms/step - accuracy: 0.9946 - loss: 0.0729

47/422		3s 9ms/step	accuracy: 0.9945	loss: 0.0732
52/422		3s 9ms/step	accuracy: 0.9944	loss: 0.0735
57/422		3s 9ms/step	accuracy: 0.9942	loss: 0.0738
66/422		3s 9ms/step	accuracy: 0.9940	loss: 0.0745
76/422		2s 8ms/step	accuracy: 0.9938	loss: 0.0750
86/422		2s 8ms/step	accuracy: 0.9937	loss: 0.0754
97/422		2s 7ms/step	accuracy: 0.9936	loss: 0.0757
107/422		2s 7ms/step	accuracy: 0.9935	loss: 0.0759
118/422		2s 7ms/step	accuracy: 0.9934	loss: 0.0761
129/422		2s 7ms/step	accuracy: 0.9934	loss: 0.0763
140/422		1s 7ms/step	accuracy: 0.9933	loss: 0.0766
151/422		1s 7ms/step	accuracy: 0.9933	loss: 0.0768
162/422		1s 6ms/step	accuracy: 0.9932	loss: 0.0769
173/422		1s 6ms/step	accuracy: 0.9932	loss: 0.0770
184/422		1s 6ms/step	accuracy: 0.9932	loss: 0.0771
195/422		1s 6ms/step	accuracy: 0.9931	loss: 0.0772
206/422		1s 6ms/step	accuracy: 0.9931	loss: 0.0773
217/422		1s 6ms/step	accuracy: 0.9931	loss: 0.0773
224/422		1s 6ms/step	accuracy: 0.9931	loss: 0.0774
235/422		1s 6ms/step	accuracy: 0.9930	loss: 0.0774
246/422		1s 6ms/step	accuracy: 0.9930	loss: 0.0775
257/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0776
268/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0776
279/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0777
290/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0777
301/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0778
312/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0778
323/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0778
334/422		0s 6ms/step	accuracy: 0.9930	loss: 0.0779
345/422		0s 6ms/step	accuracy: 0.9929	loss: 0.0779

355/422 ————— 0s 6ms/step – accuracy: 0.9929 – loss: 0.0779

366/422 ————— 0s 6ms/step – accuracy: 0.9929 – loss: 0.0779

377/422 ————— 0s 5ms/step – accuracy: 0.9929 – loss: 0.0780

388/422 ————— 0s 5ms/step – accuracy: 0.9929 – loss: 0.0780

399/422 ————— 0s 5ms/step – accuracy: 0.9929 – loss: 0.0780

410/422 ————— 0s 5ms/step – accuracy: 0.9929 – loss: 0.0780

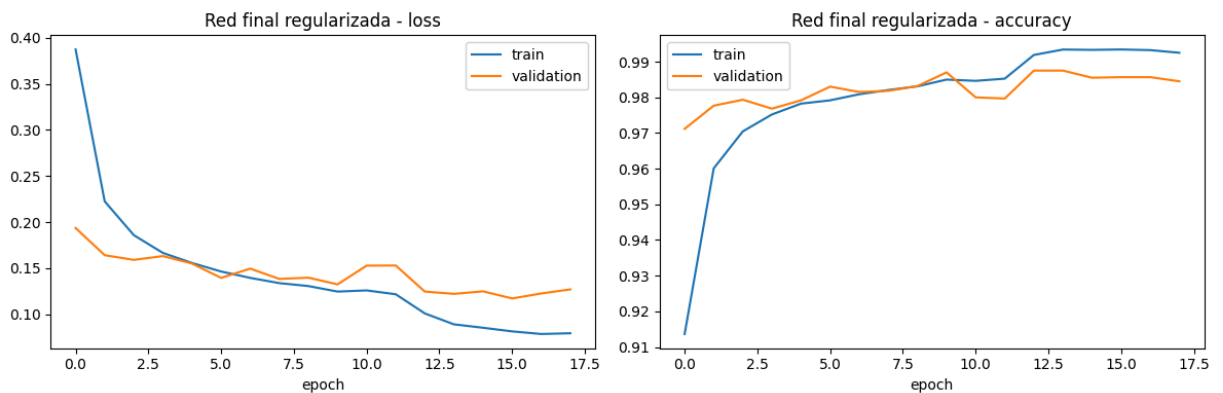
421/422 ————— 0s 5ms/step – accuracy: 0.9929 – loss: 0.0781

Epoch 18: ReduceLRonPlateau reducing learning rate to 0.0002500000118743628.

422/422 ————— 2s 6ms/step – accuracy: 0.9925 – loss: 0.0794 – val_accuracy: 0.9845 – val_loss: 0.1270 – learning_rate: 5.0000e-04

Epoch 18: early stopping

Restoring model weights from the end of the best epoch: 13.



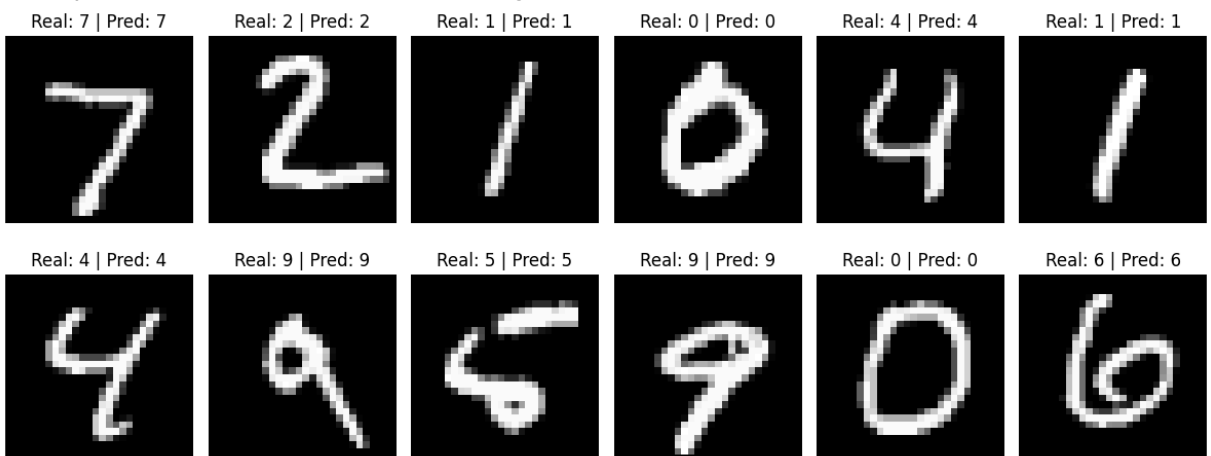
Mejor validation accuracy: 0.9875

Mejor validation loss: 0.1171

Test accuracy final: 0.9840

Test loss final: 0.1265

Tiempo de entrenamiento: 41.6 segundos



	pred_ 0	pred_ 1	pred_ 2	pred_ 3	pred_ 4	pred_ 5	pred_ 6	pred_ 7	pred_ 8	pred_ 9
real_ 0	970	1	0	0	0	0	2	1	4	2
real_ 1	0	1128	2	1	0	0	1	1	2	0
real_ 2	2	0	1016	4	0	0	0	8	2	0
real_ 3	1	0	2	996	0	2	0	5	1	3
real_ 4	0	0	4	1	956	0	1	2	0	18
real_ 5	2	0	0	9	0	869	2	3	5	2
real_ 6	4	2	1	1	2	2	942	0	4	0
real_ 7	1	4	4	2	0	0	0	1012	2	3
real_ 8	3	0	0	0	2	2	0	5	959	3
real_ 9	2	2	0	3	2	2	0	2	4	992

	precision	recall	f1-score	support
0	0.9848	0.9898	0.9873	980
1	0.9921	0.9938	0.9930	1135
2	0.9874	0.9845	0.9859	1032
3	0.9794	0.9861	0.9827	1010
4	0.9938	0.9735	0.9835	982
5	0.9909	0.9742	0.9825	892
6	0.9937	0.9833	0.9885	958
7	0.9740	0.9844	0.9792	1028
8	0.9756	0.9846	0.9801	974
9	0.9697	0.9832	0.9764	1009
accuracy			0.9840	10000
macro avg	0.9841	0.9837	0.9839	10000
weighted avg	0.9841	0.9840	0.9840	10000

Conclusión del problema 12.1:

La red final combina las decisiones que han dado mejores resultados en los apartados anteriores: activación `ReLU`, inicialización `glorot_uniform`, optimizador `Adam`, varias capas densas, `BatchNormalization`, `Dropout`, regularización L2, reducción automática de la tasa de aprendizaje y `EarlyStopping`. La idea es dar al modelo suficiente capacidad para aprender los patrones de los dígitos, pero controlando el sobreajuste. La mejor `validation accuracy` mostrada arriba supera claramente el 95%, y la evaluación con `predict()` permite comprobar el comportamiento sobre ejemplos reales del conjunto de test.